

CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Shanti Mantra

Lyrics in Sanskrit

Meaning in English

ॐ सह नाववतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

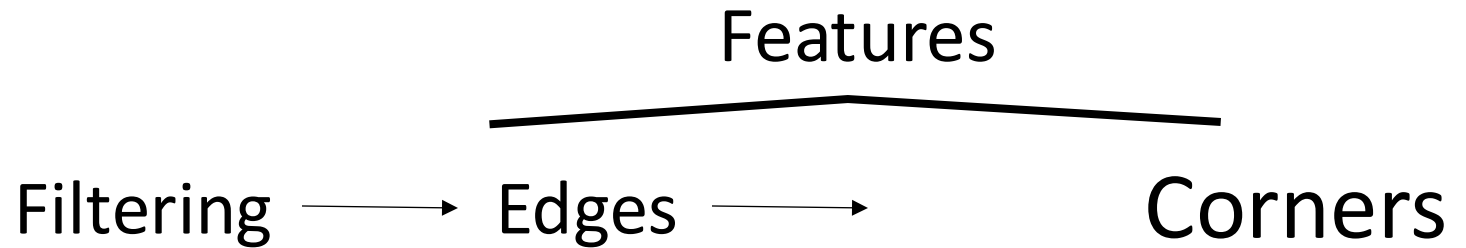
मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)

Interest Points & Harris Corner Detection



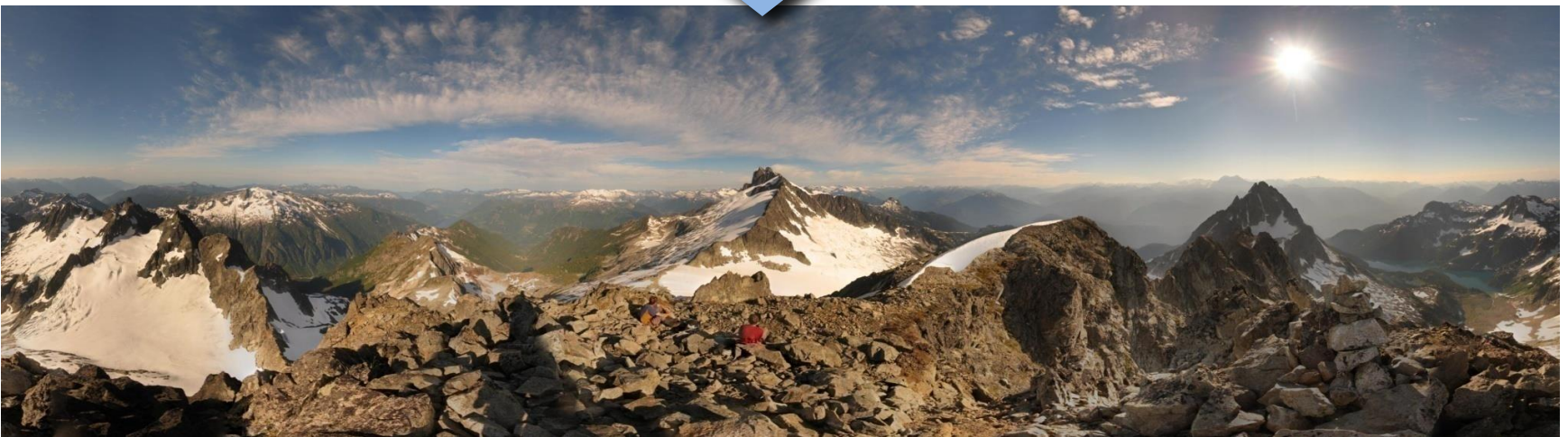
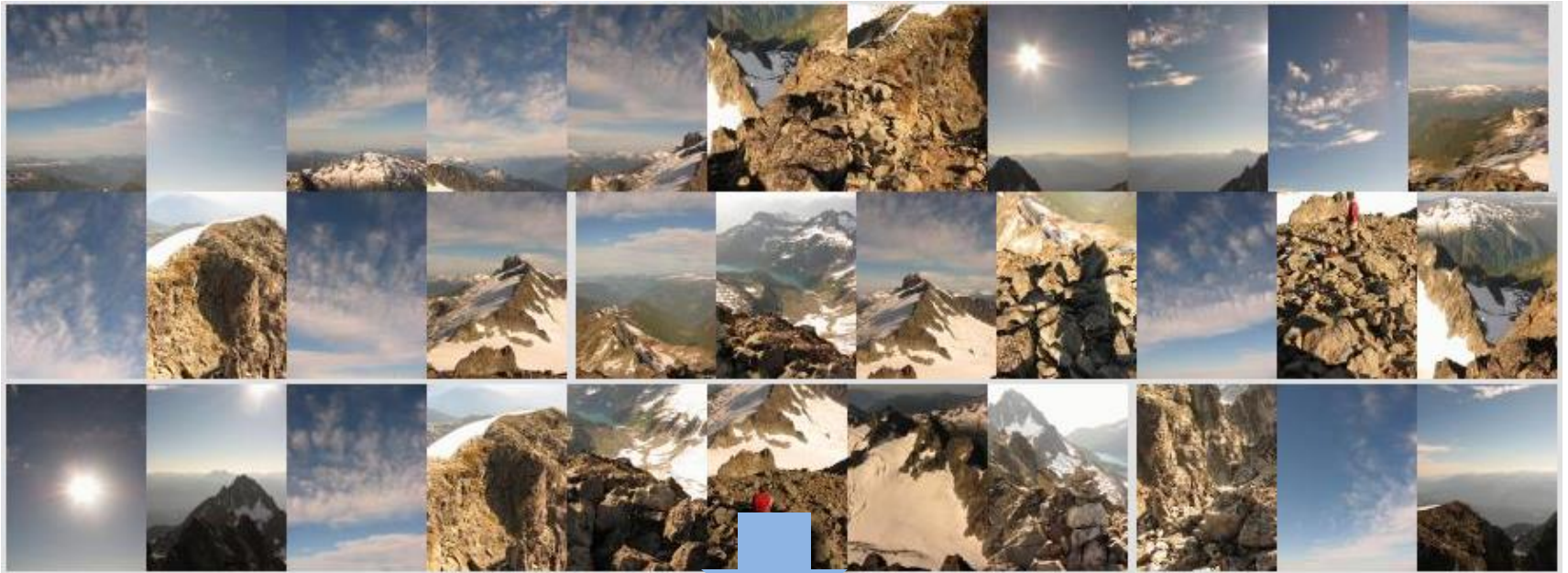
Corners are also called
interest points or key points.

“Local features”

Example: structure from motion



Motivation: Automatic panoramas



Motivation: Automatic panoramas



GigaPan

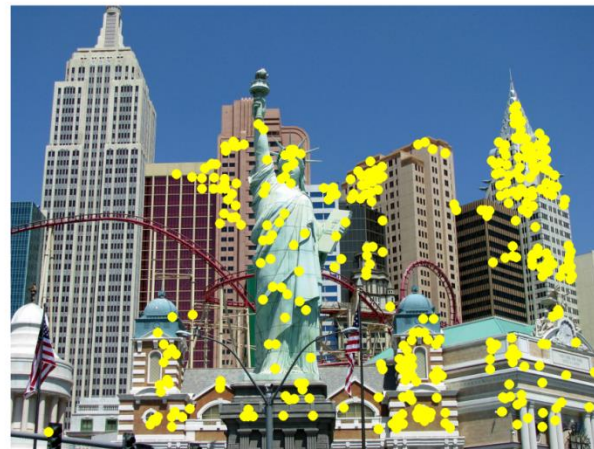
<http://gigapan.com/>

Google Gigapixels

<https://artsandculture.google.com/project/gigapixels>

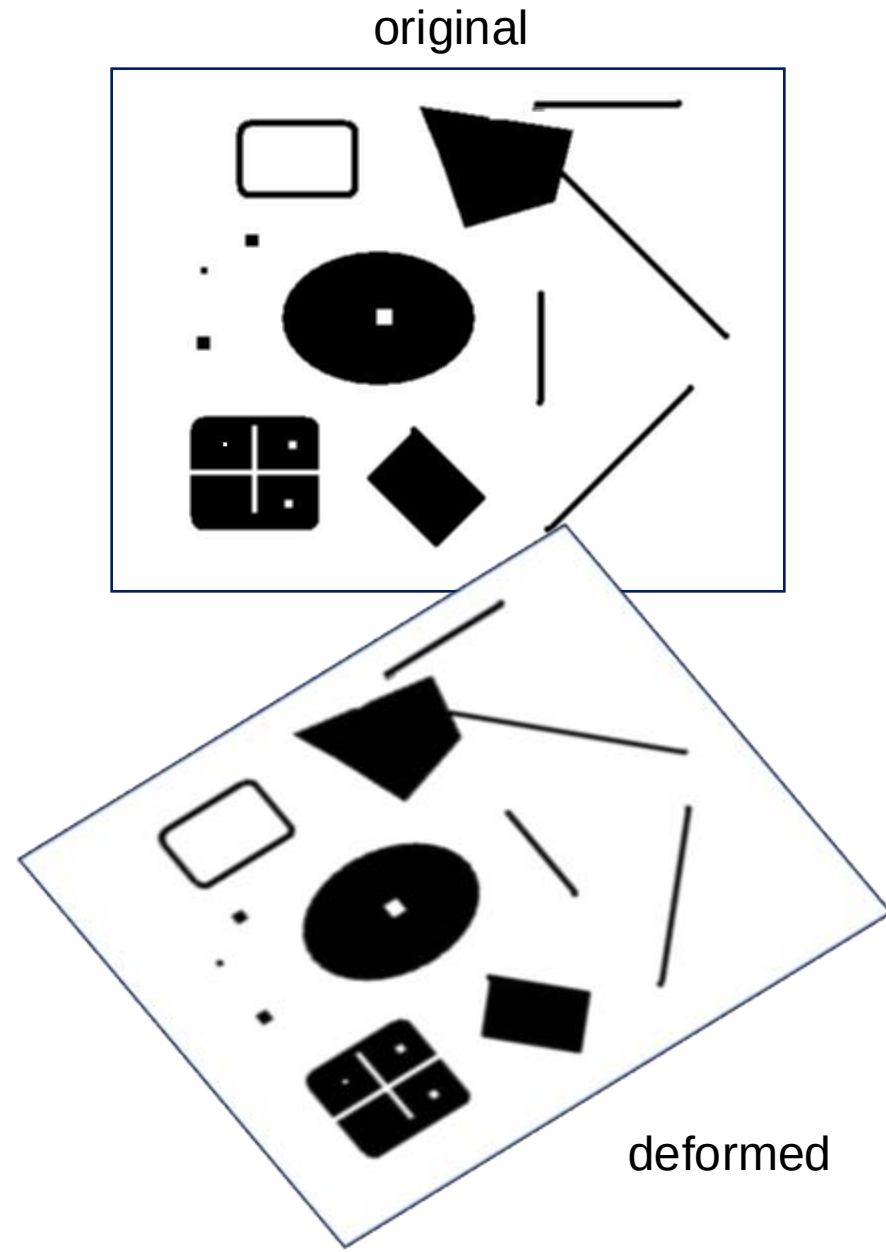
Fundamental to Applications

- Feature points are used for:
 - Image alignment
 - 3D reconstruction
 - Motion tracking (robots, drones, AR)
 - Indexing and database retrieval
 - Object recognition



This class: interest points

- Suppose you have to click on some point, go away and come back after I deform the image, and click on the same points again.
 - Which points would you choose?

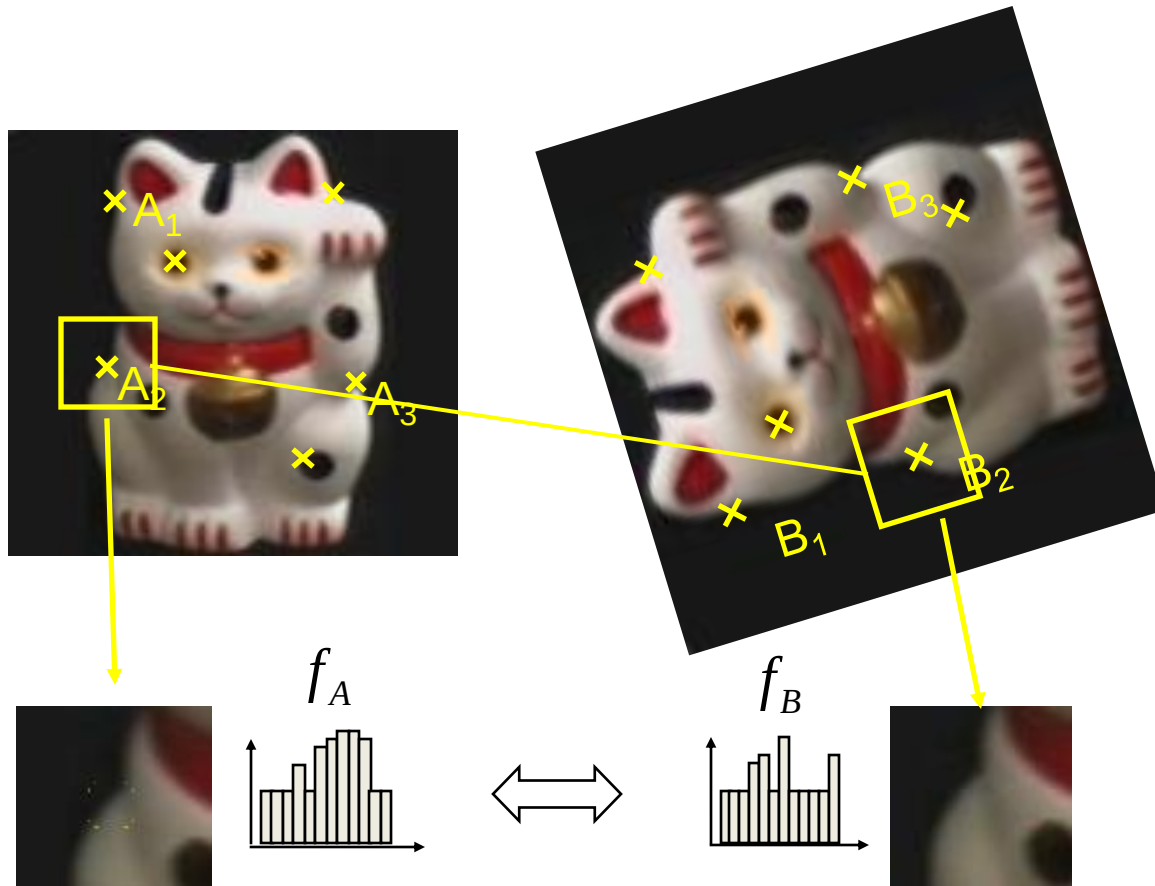


Goals for Keypoints



Detect points that are *repeatable* and *distinctive*

Overview of Keypoint Matching



1. Find a set of distinctive key-points

2. Define a region around each keypoint

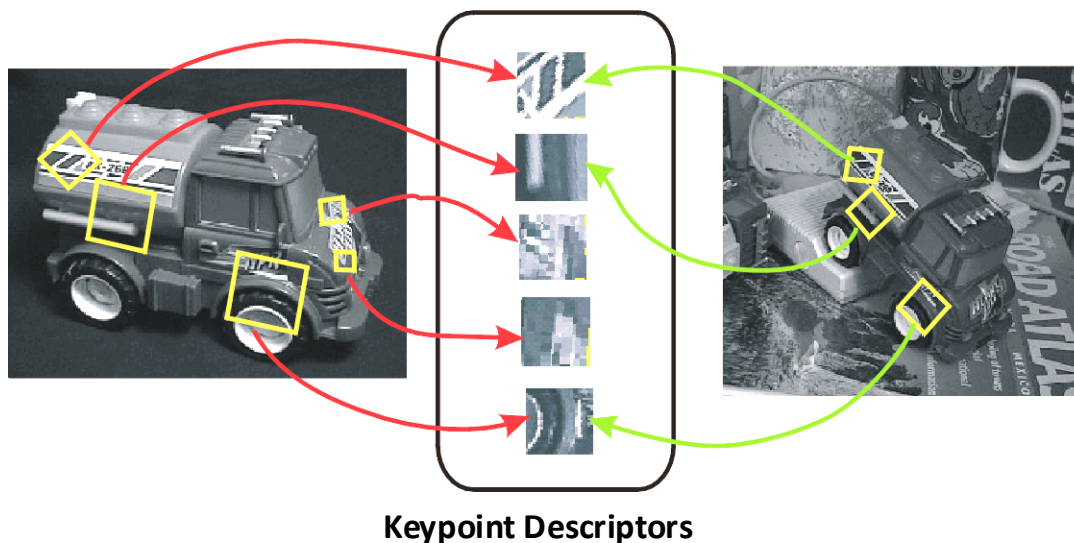
3. Compute a local descriptor from the normalized region

4. Match local descriptors

Correspondence in Object Detection

Finding *distinctive* and *repeatable* feature points can be difficult when we want our features to be invariant to large transformations:

- geometric variation (translation, rotation, scale, perspective)
- appearance variation (reflectance, illumination)



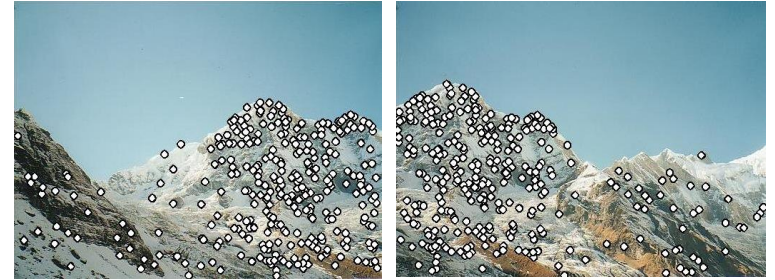
Example: Panorama Stitching

We have two images –
how do we overlay them?

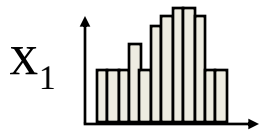


Local features: main components

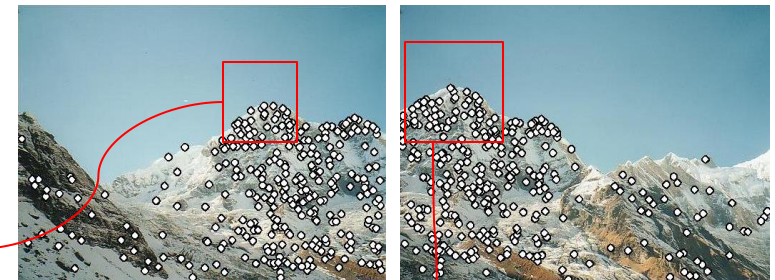
- 1) Detection:
Find a set of distinctive key points.



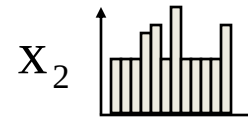
- 2) Description:
Extract feature descriptor around each interest point as vector.



$$\mathbf{x}_1 = [x_1^{(1)}, \dots, x_d^{(1)}]$$

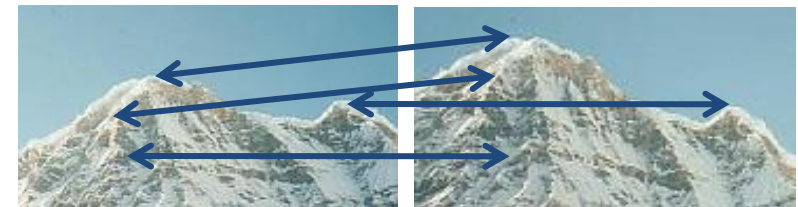


- 3) Matching:
Compute distance between feature vectors to find correspondence.



$$\mathbf{x}_2 = [x_1^{(2)}, \dots, x_d^{(2)}]$$

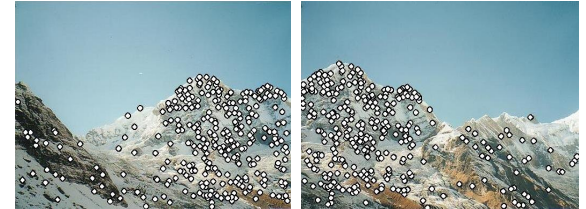
$$d(\mathbf{x}_1, \mathbf{x}_2) < T$$



Local Feature Matching: Main Components

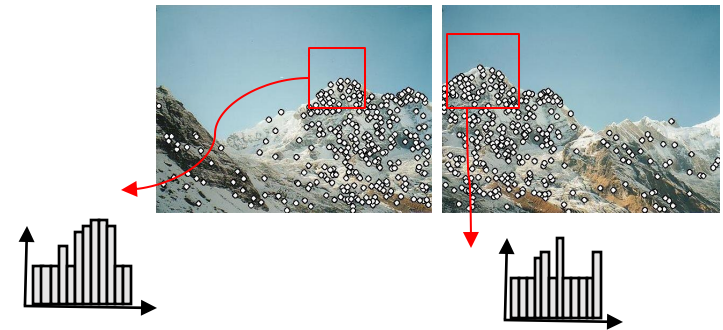
1) Detection:

Find a set of distinctive features (e.g., key points)



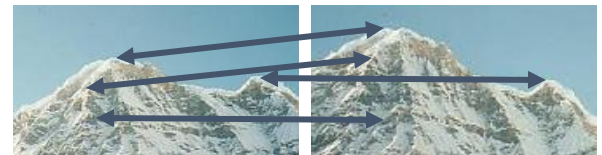
2) Description:

Extract feature descriptor around each interest point as vector.

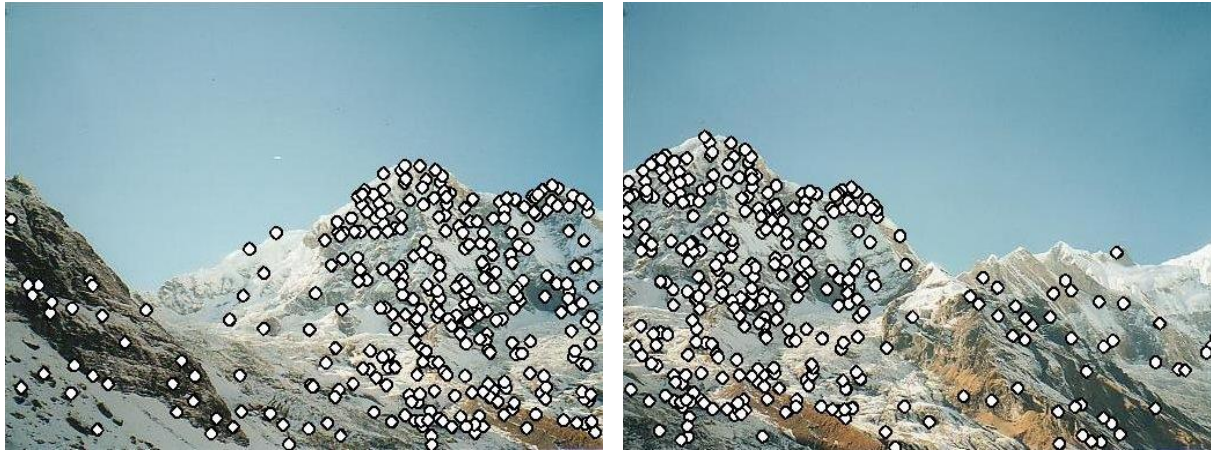


3) Matching:

Compute distance between feature vectors to find correspondence.



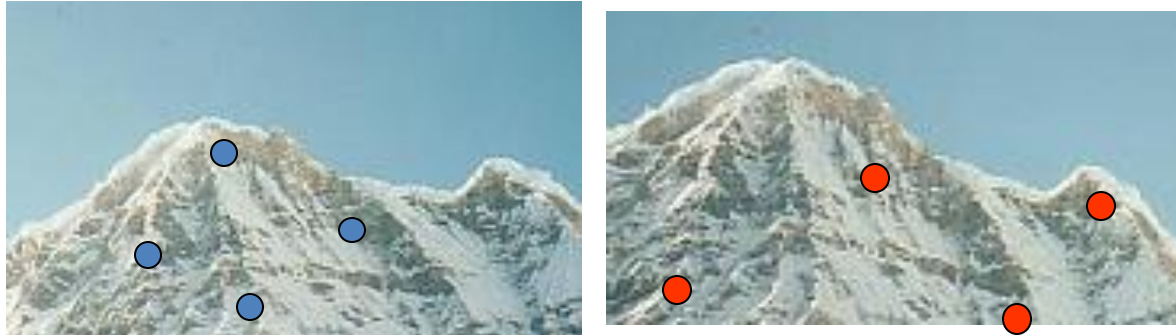
Characteristics of good features



- **Repeatability**
 - The same feature can be found in several images despite geometric and photometric transformations
- **Saliency**
 - Each feature is distinctive
- **Compactness and efficiency**
 - Many fewer features than image pixels
- **Locality**
 - A feature occupies a relatively small area of the image; robust to clutter and occlusion

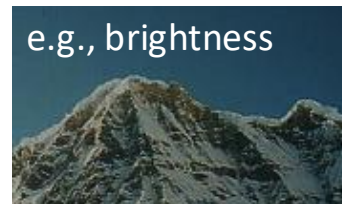
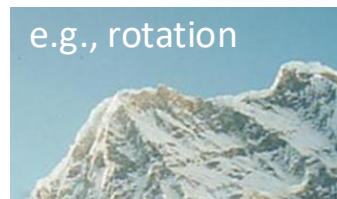
Goal: Repeatability

We want to detect (at least some of) the same points in both images.



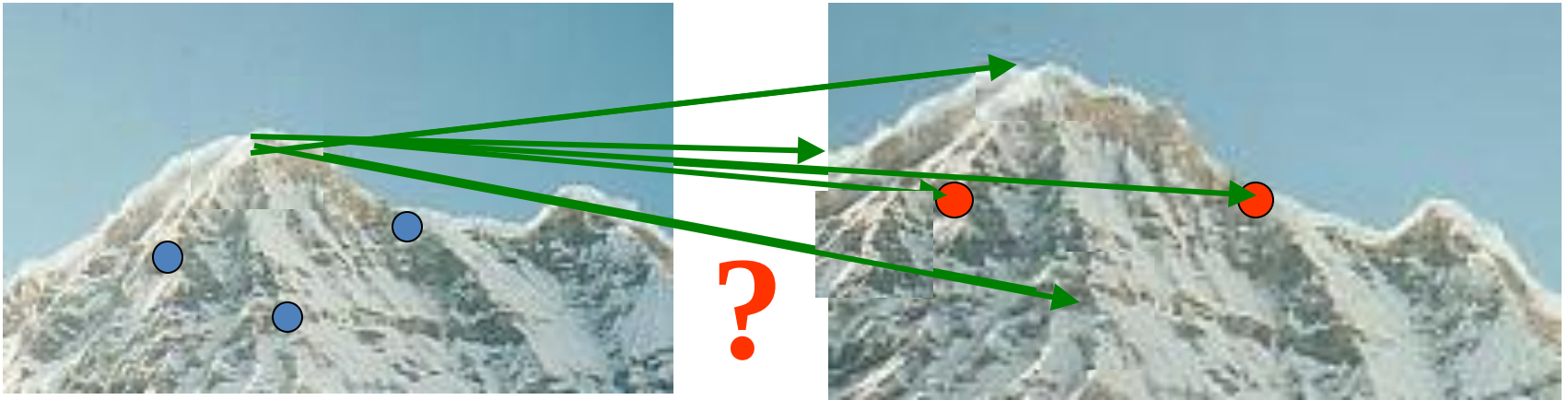
With these points, there's no chance to find true matches!

Under geometric and photometric variations.



Goal: descriptor distinctiveness

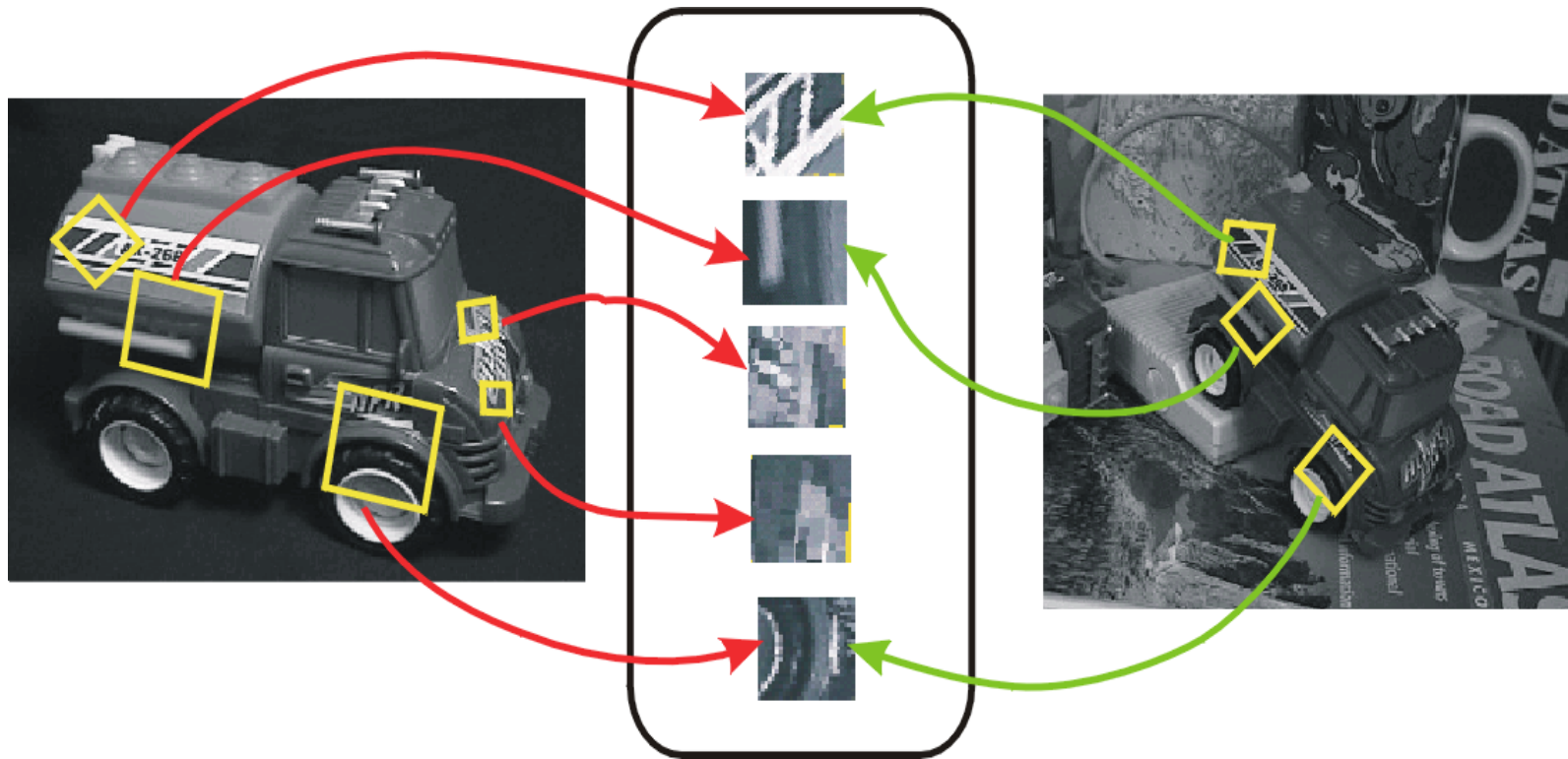
- We want to be able to reliably determine which point goes with which.



- Must provide some invariance to geometric and photometric differences between the two views.

Invariant Local Features

- Image content is transformed into local feature coordinates that are invariant to translation, rotation, scale, and other imaging parameters

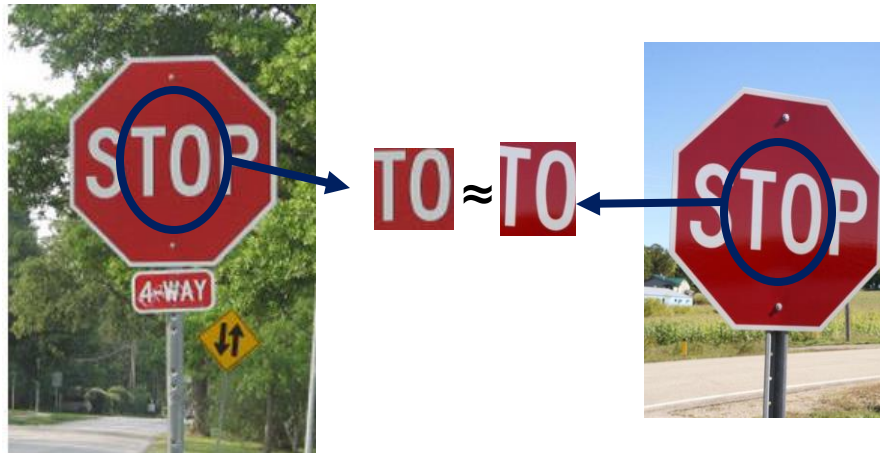


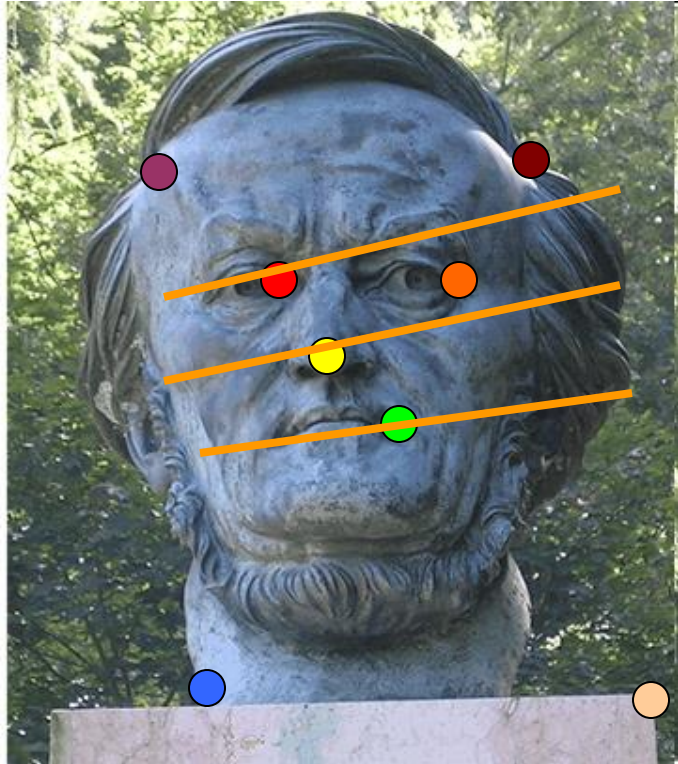
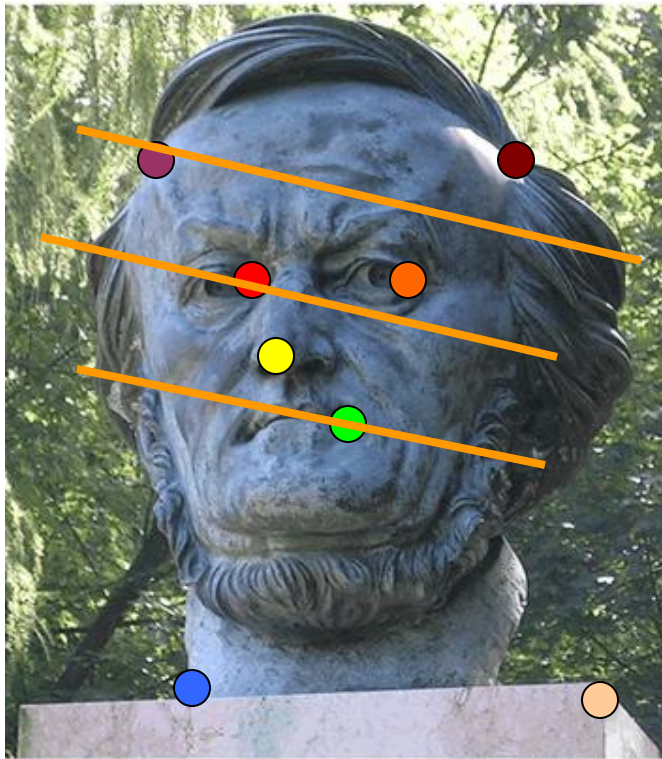
Features Descriptors

Correspondence

Matching points, patches, edges, or regions across images.

Sparse or local correspondence vs. dense (at every pixel)





Goal: Compactness and Efficiency

We want the representation to be as small and as fast as possible

- Much smaller than a whole image

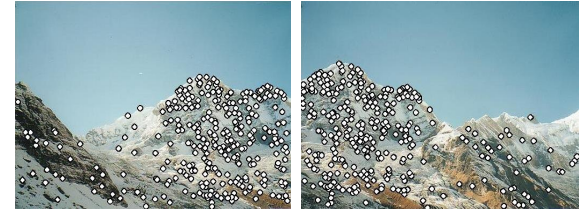
Sometimes, we'd like to run the detection procedure *independently* per image

- Match just the compact descriptors for speed.
- *Difficult!* We don't get to see 'the other image' at match time, e.g., object detection.

Local Feature Matching: Main Components

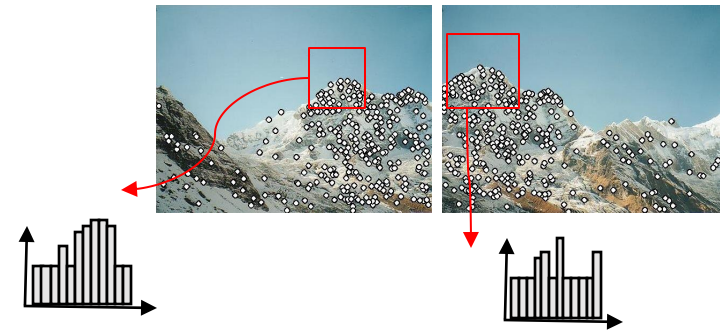
1) Detection:

Find a set of distinctive features (e.g., key points)



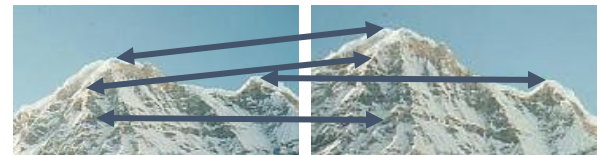
2) Description:

Extract feature descriptor around each interest point as vector.



3) Matching:

Compute distance between feature vectors to find correspondence.



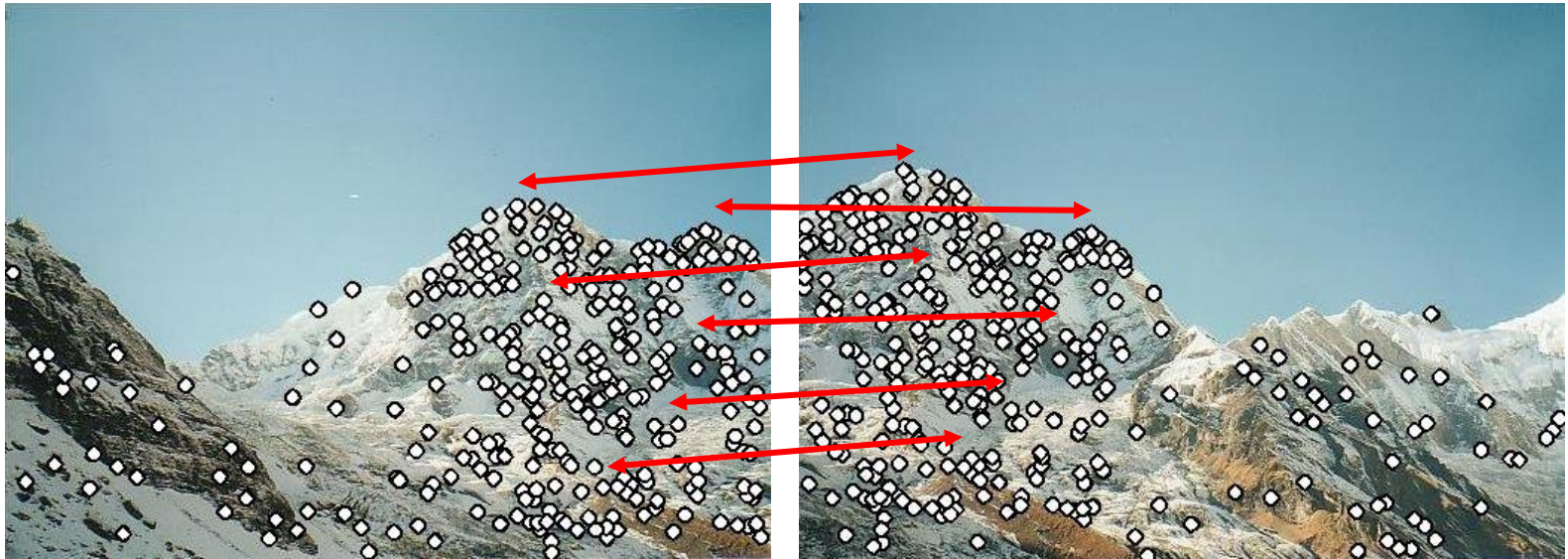
Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract features

Step 2: match features

Why extract features?

- Motivation: panorama stitching
 - We have two images – how do we combine them?



Step 1: extract features

Step 2: match features

Step 3: align images

Feature Matching



More motivation...

Feature points are used for:

- Image alignment (e.g., mosaics)
- 3D reconstruction
- Motion tracking
- Object recognition
- Indexing and database retrieval
- Robot navigation
- ... other

Many Existing Detectors Available

Hessian & Harris

Laplacian, DoG

Harris-/Hessian-Laplace

Harris-/Hessian-Affine

EBR and IBR

MSER

Salient Regions

Others...

[Beaudet '78], [Harris '88]

[Lindeberg '98], [Lowe 1999]

[Mikolajczyk & Schmid '01]

[Mikolajczyk & Schmid '04]

[Tuytelaars & Van Gool '04]

[Matas '02]

[Kadir & Brady '01]

Detectors we will mostly focus on:

- For Corners
 - Harris Corners

Corner Detection: Basic Idea

We do not know which other image locations the feature will end up being matched against.

But we can compute how stable a location is in appearance with respect to small variations in its position.

Strategy: Compare image patch against local neighbors.

Want uniqueness

Look for image regions that are unusual

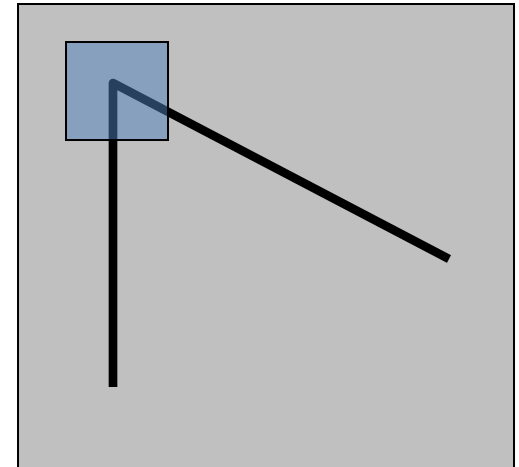
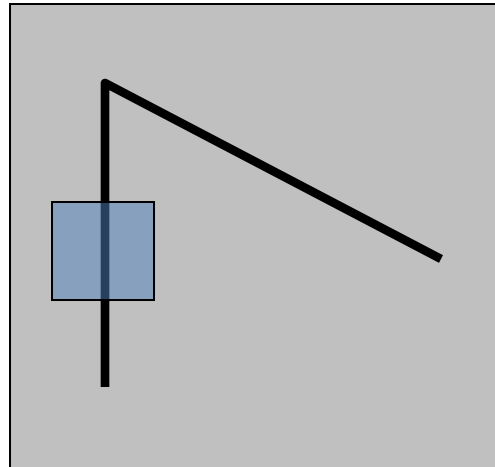
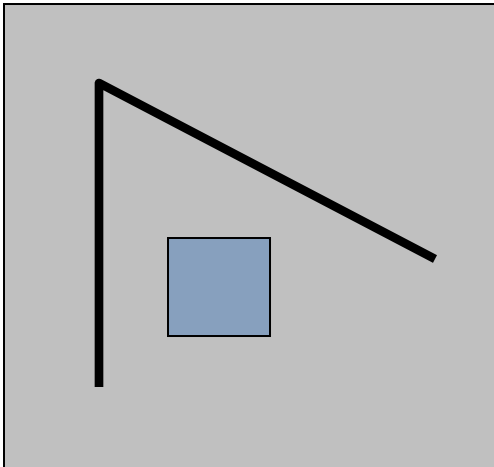
- Lead to unambiguous matches in other images

How to define “unusual”?

Local measures of uniqueness

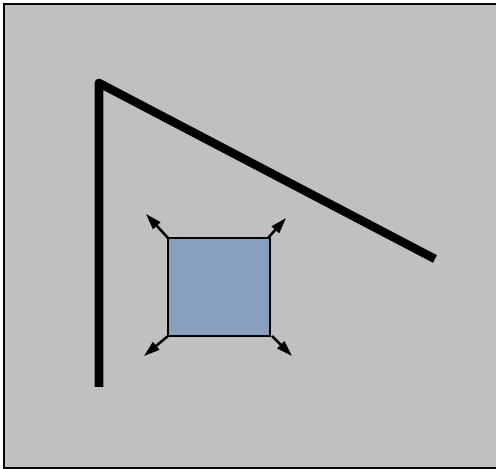
Suppose we only consider a small window of pixels

- What defines whether a feature is a good or bad candidate?

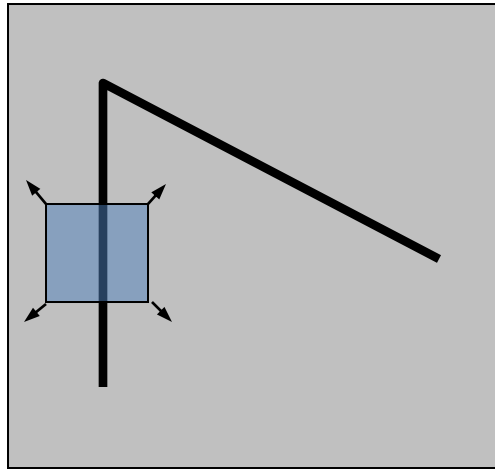


Local measure of feature uniqueness

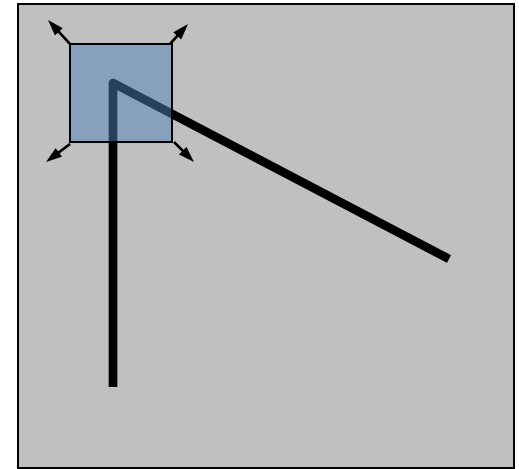
- How does the window change when you shift it?
- Shifting the window in any direction causes a big change



“flat” region:
no change in all
directions



“edge”:
no change along the
edge direction



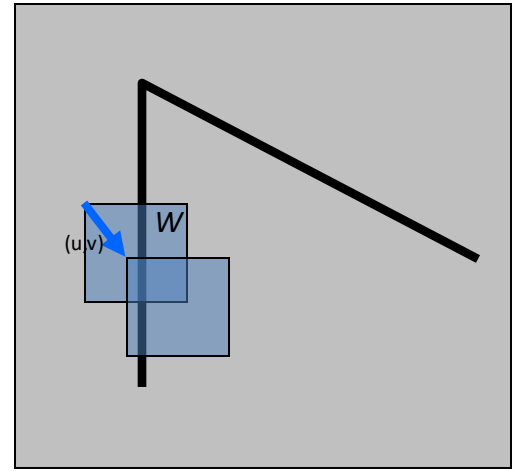
“corner”:
significant change in
all directions

Harris corner detection: the math

Consider shifting the window W by (u, v)

- how do the pixels in W change?
- compare each pixel before and after by summing up the squared differences (SSD)
- this defines an SSD “error” $E(u, v)$:

$$E(u, v) = \sum_{(x, y) \in W} [I(x + u, y + v) - I(x, y)]^2$$



- We are happy if this error is high
- Slow to compute exactly for each pixel and each offset (u, v)

Corner Detection by Auto-correlation

Change in appearance of window $w(x,y)$ for shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

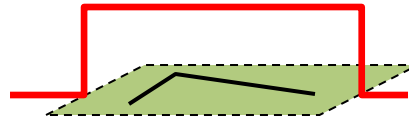
Window
function

Shifted
intensity

Intensity

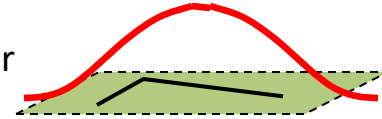
Also called 'sum of squared differences' or SSD

Window function $w(x,y) =$



1 in window, 0 outside

or

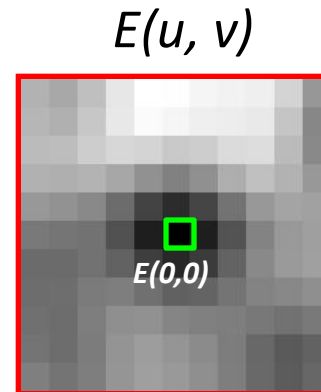
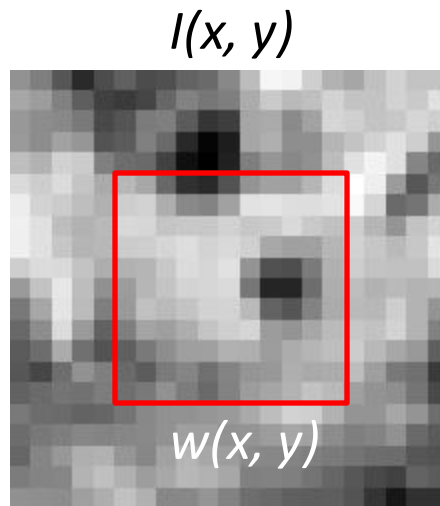


Gaussian

Corner Detection by Auto-correlation

Change in appearance of window $w(x,y)$ for shift $[u,v]$:

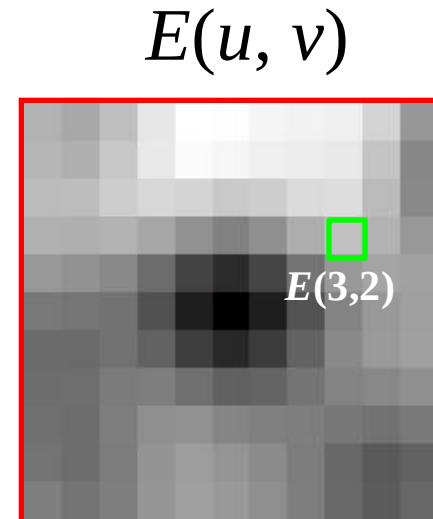
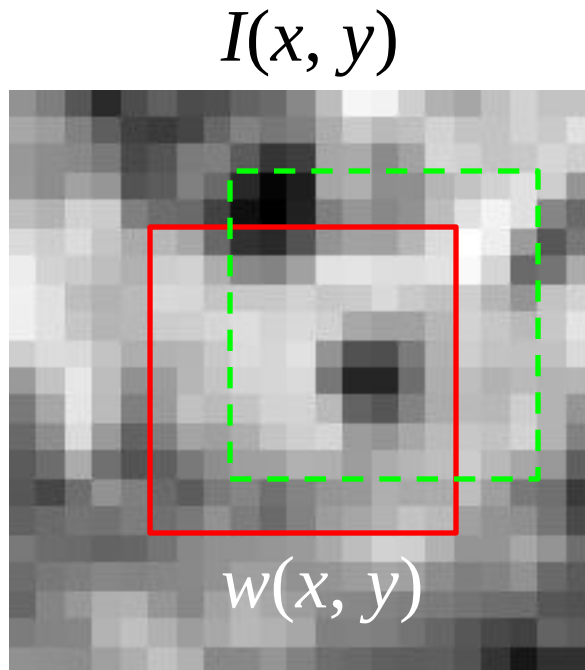
$$E(u, v) = \sum_{x,y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$



Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

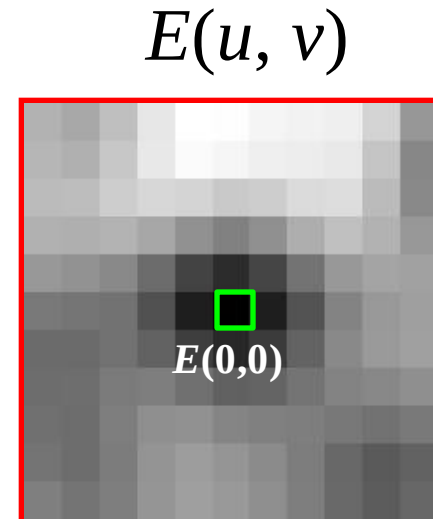
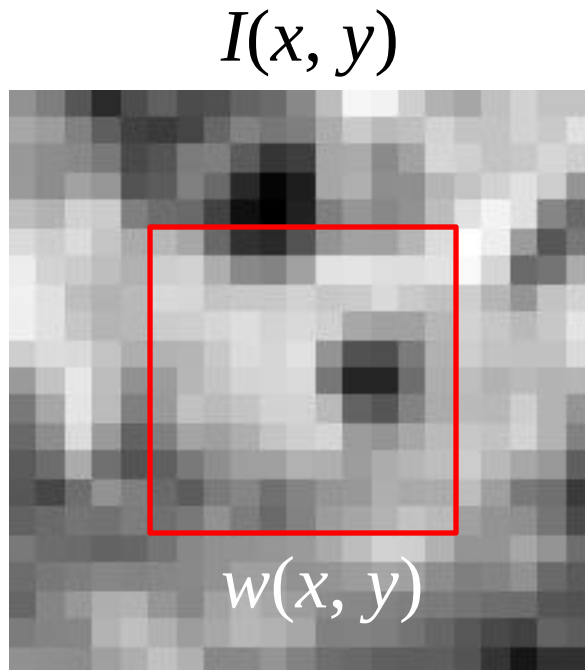
$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$



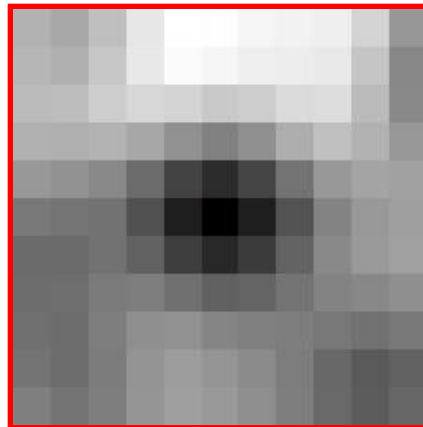
Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

We want to find out how this function behaves for small shifts

$E(u, v)$



Think-Pair-Share

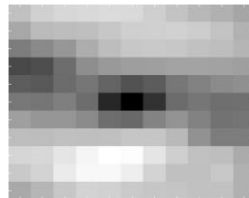
$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

Correspond the three red crosses to (b,c,d).

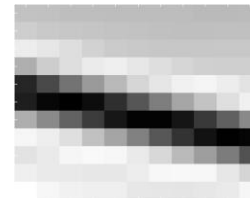


$E(u, v)$

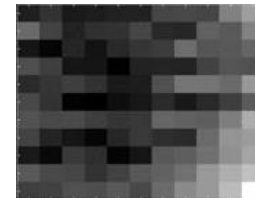
b)



c)

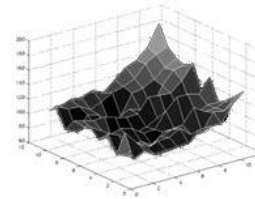
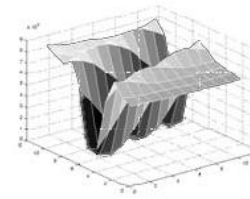
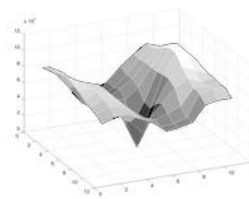


d)



$E(u, v)$

As a surface



Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

We want to find out how this function behaves for small shifts

But this is very slow to compute naively.

$O(\text{window_width}^2 * \text{shift_range}^2 * \text{image_width}^2)$

Corner Detection: Mathematics

$$O(\text{window_width}^2 * \text{shift_range}^2 * \text{image_width}^2) \\ = O(W^2 S^2 M^2)$$

- $W = 11$ (window size is 11×11)
- $S = 11$ (shift range is 11×11)
- $M = 600$ (image size is 600×600)

Computing the number of operations:

$$\begin{aligned} &O(11^2 \times 11^2 \times 600^2) \\ &= (121) \times (121) \times (360000) \\ &= 5.2 \text{ billion operations} \end{aligned}$$

Since we are doing this for each pixel, we divide by the total number of pixels (600×600) to get:

$$\frac{5.2 \text{ billion}}{360000} \approx 14.6k \text{ operations per pixel.}$$

Why is This Slow?

- This direct computation is impractical for large images because it requires evaluating millions of shifts across a large number of windows.
- Instead, efficient methods like **Harris Corner Detection** approximate $E(u,v)$ using the **structure tensor** and avoid brute-force shifting.

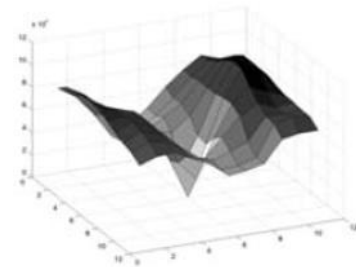
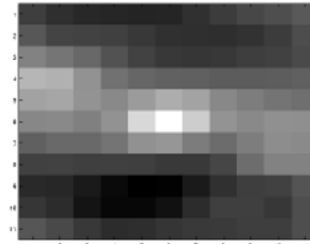
Corner Detection by Auto-correlation

Change in appearance of window $w(x,y)$ for shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

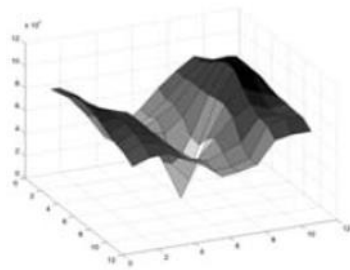
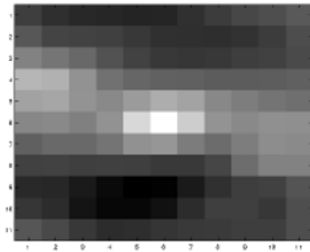
We want to discover how E behaves for small shifts

But we know the response in E that we are looking for – strong peak.

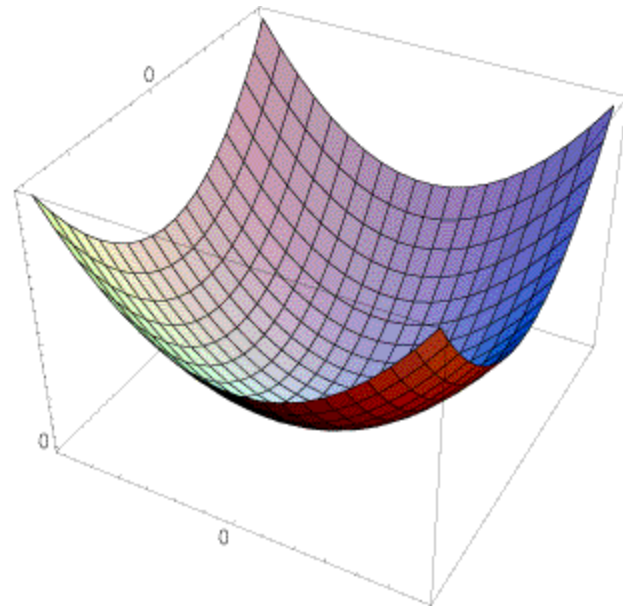


Strategy

Approximate $E(u,v)$ locally by a quadratic surface, and look for that instead.



$\approx$



Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

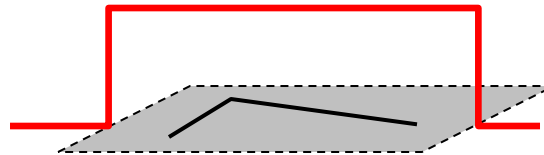
$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

Window
function

Shifted
intensity

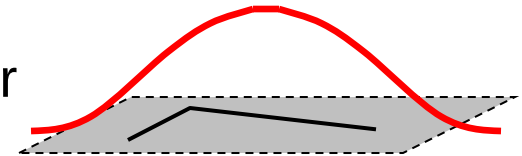
Intensity

Window function $w(x,y) =$



1 in window, 0 outside

or



Gaussian

Corner Detection: Mathematics

Change in appearance of window $w(x,y)$
for the shift $[u,v]$:

$$E(u,v) = \sum_{x,y} w(x,y) [I(x+u, y+v) - I(x,y)]^2$$

We want to find out how this function behaves for small shifts

Recall Taylor series expansion. A function f can be approximated around point a as

$$f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

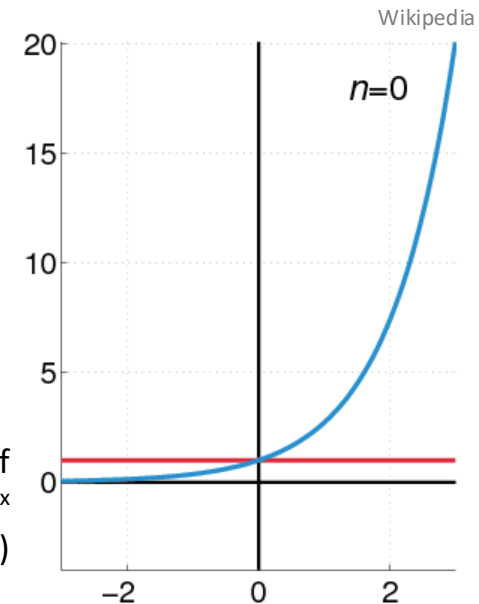
Recap: Taylor Series Expansion

A function f can be represented by an infinite series of its derivatives at a single point a :

$$f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

As we care about window centered, we set $a = 0$
(MacLaurin series)

Approximation of
 $f(x) = e^x$
centered at $f(0)$



Local quadratic approximation of $E(u,v)$ in the neighborhood of $(0,0)$ is given by the *second-order Taylor expansion*:

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Notation: partial derivative



$E(u, v)$ and its derivatives

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

$$E_u(u, v) = \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_x(x+u, y+v)$$

$$\begin{aligned} E_{uu}(u, v) &= \sum_{x, y} 2w(x, y) I_x(x+u, y+v) I_x(x+u, y+v) \\ &\quad + \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_{xx}(x+u, y+v) \end{aligned}$$

$$\begin{aligned} E_{uv}(u, v) &= \sum_{x, y} 2w(x, y) I_y(x+u, y+v) I_x(x+u, y+v) \\ &\quad + \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_{xy}(x+u, y+v) \end{aligned}$$

$E(u, v)$ and its derivatives

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

$$E_u(u, v) = \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_x(x+u, y+v)$$

$$E_{uu}(u, v) = \sum_{x, y} 2w(x, y) I_x(x+u, y+v) I_x(x+u, y+v) \\ + \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_{xx}(x+u, y+v)$$

$$E_{uv}(u, v) = \sum_{x, y} 2w(x, y) I_y(x+u, y+v) I_x(x+u, y+v) \\ + \sum_{x, y} 2w(x, y) [I(x+u, y+v) - I(x, y)] I_{xy}(x+u, y+v)$$

$$\left. \frac{\partial E}{\partial u} \right|_{(0,0)} = \left. \frac{\partial E}{\partial v} \right|_{(0,0)} = 0,$$

$$\left. \frac{\partial^2 E}{\partial u^2} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_x^2(x, y),$$

$$\left. \frac{\partial^2 E}{\partial v^2} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_y^2(x, y),$$

$$\left. \frac{\partial^2 E}{\partial u \partial v} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_x(x, y) I_y(x, y)$$

$E(u, v)$ and its derivatives

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

$$\left. \frac{\partial E}{\partial u} \right|_{(0,0)} = \left. \frac{\partial E}{\partial v} \right|_{(0,0)} = 0,$$

$$\left. \frac{\partial^2 E}{\partial u^2} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_x^2(x, y),$$

$$\left. \frac{\partial^2 E}{\partial v^2} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_y^2(x, y),$$

$$\left. \frac{\partial^2 E}{\partial u \partial v} \right|_{(0,0)} = 2 \sum_{x, y} w(x, y) I_x(x, y) I_y(x, y)$$

$$E(0,0) = 0$$

$$E_u(0,0) = 0$$

$$E_v(0,0) = 0$$

$$E_{uu}(0,0) = \sum_{x, y} 2w(x, y) I_x(x, y) I_x(x, y)$$

$$E_{vv}(0,0) = \sum_{x, y} 2w(x, y) I_y(x, y) I_y(x, y)$$

$$E_{uv}(0,0) = \sum_{x, y} 2w(x, y) I_x(x, y) I_y(x, y)$$

Corner Detection: Mathematics

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Second-order Taylor expansion of $E(u, v)$ about $(0, 0)$:

$$E(u, v) \approx E(0, 0) + [u \ v] \begin{bmatrix} E_u(0, 0) \\ E_v(0, 0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0, 0) & E_{uv}(0, 0) \\ E_{uv}(0, 0) & E_{vv}(0, 0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E(0, 0) = 0$$

$$E_u(0, 0) = 0$$

$$E_v(0, 0) = 0$$

$$E_{uu}(0, 0) = \sum_{x, y} 2w(x, y) I_x(x, y) I_x(x, y)$$

$$E_{vv}(0, 0) = \sum_{x, y} 2w(x, y) I_y(x, y) I_y(x, y)$$

$$E_{uv}(0, 0) = \sum_{x, y} 2w(x, y) I_x(x, y) I_y(x, y)$$

Local quadratic approximation of $E(u,v)$ in the neighborhood of $(0,0)$ is given by the *second-order Taylor expansion*:

$$E(u,v) \approx E(0,0) + [u \ v] \begin{bmatrix} E_u(0,0) \\ E_v(0,0) \end{bmatrix} + \frac{1}{2} [u \ v] \begin{bmatrix} E_{uu}(0,0) & E_{uv}(0,0) \\ E_{uv}(0,0) & E_{vv}(0,0) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$


Ignore function
value; set to 0


Ignore first
derivative,
set to 0


Just look at shape of
second derivative
(2D quadratic surface)

Corner Detection: Mathematics

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

Second-order Taylor expansion of $E(u, v)$ about $(0, 0)$:

$$E(u, v) \approx [u \ v] \begin{bmatrix} \sum_{x, y} w(x, y) I_x^2(x, y) & \sum_{x, y} w(x, y) I_x(x, y) I_y(x, y) \\ \sum_{x, y} w(x, y) I_x(x, y) I_y(x, y) & \sum_{x, y} w(x, y) I_y^2(x, y) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Corner Detection: Mathematics

Second-order Taylor expansion of $E(u,v)$ about $(0,0)$:

$$E(u,v) \approx [u \ v] \begin{bmatrix} \sum_{x,y} w(x,y) I_x^2(x,y) & \sum_{x,y} w(x,y) I_x(x,y) I_y(x,y) \\ \sum_{x,y} w(x,y) I_x(x,y) I_y(x,y) & \sum_{x,y} w(x,y) I_y^2(x,y) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

$$E(u,v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Corner Detection: Mathematics

The quadratic approximation simplifies to

$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is a *second moment matrix* computed from image derivatives:

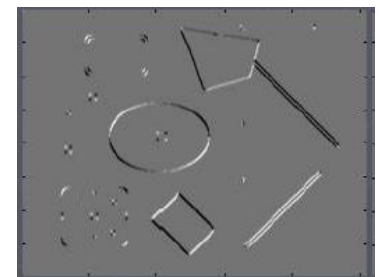
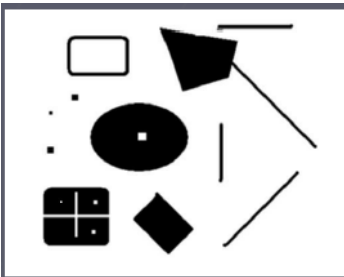
$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

$$M = \begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} = \sum \begin{bmatrix} I_x \\ I_y \end{bmatrix} [I_x \ I_y] = \sum \nabla I (\nabla I)^T$$

Corners as distinctive interest points

$$M = \sum w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

2 x 2 matrix of image derivatives (averaged in neighborhood of a point).



Notation:

$$I_x \Leftrightarrow \frac{\partial I}{\partial x}$$

$$I_y \Leftrightarrow \frac{\partial I}{\partial y}$$

$$I_x I_y \Leftrightarrow \frac{\partial I}{\partial x} \frac{\partial I}{\partial y}$$

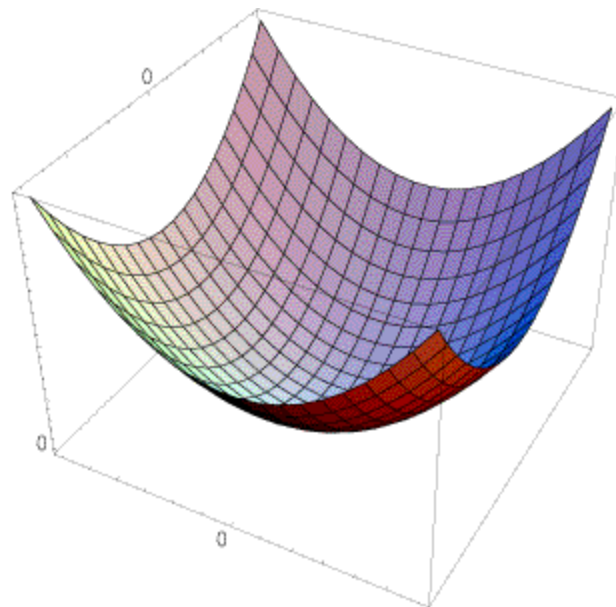
Interpreting the Second Moment Matrix

$E(u,v)$ is locally approximated by a quadratic surface.

Let's try to understand how its shape relates to M .

$$E(u,v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

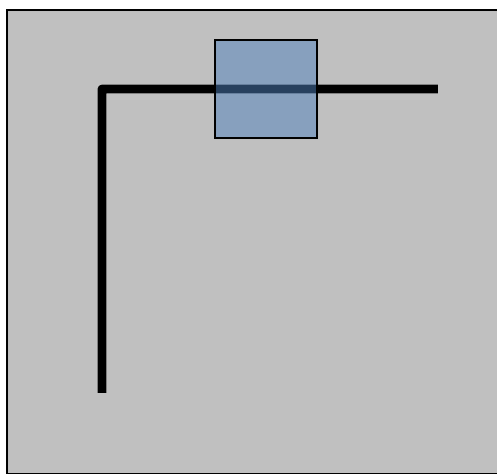


$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_H \begin{bmatrix} u \\ v \end{bmatrix}$$

$$A = \sum_{(x,y) \in W} I_x^2$$

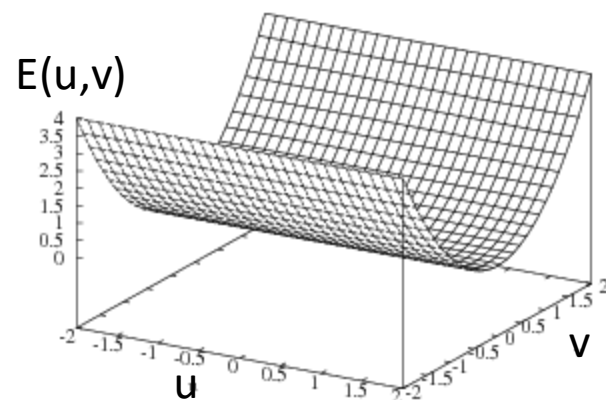
$$B = \sum_{(x,y) \in W} I_x I_y$$

$$C = \sum_{(x,y) \in W} I_y^2$$



Horizontal edge: $I_x = 0$

$$H = \begin{bmatrix} 0 & 0 \\ 0 & C \end{bmatrix}$$

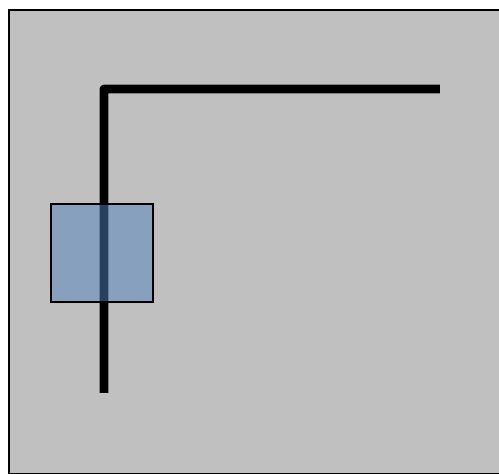


$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} \underbrace{\begin{bmatrix} A & B \\ B & C \end{bmatrix}}_H \begin{bmatrix} u \\ v \end{bmatrix}$$

$$A = \sum_{(x,y) \in W} I_x^2$$

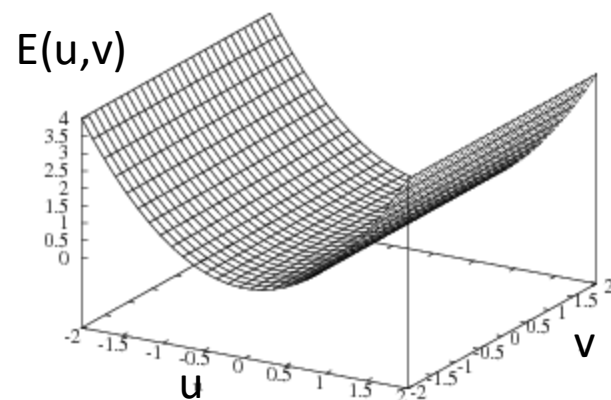
$$B = \sum_{(x,y) \in W} I_x I_y$$

$$C = \sum_{(x,y) \in W} I_y^2$$



Vertical edge: $I_y = 0$

$$H = \begin{bmatrix} A & 0 \\ 0 & 0 \end{bmatrix}$$

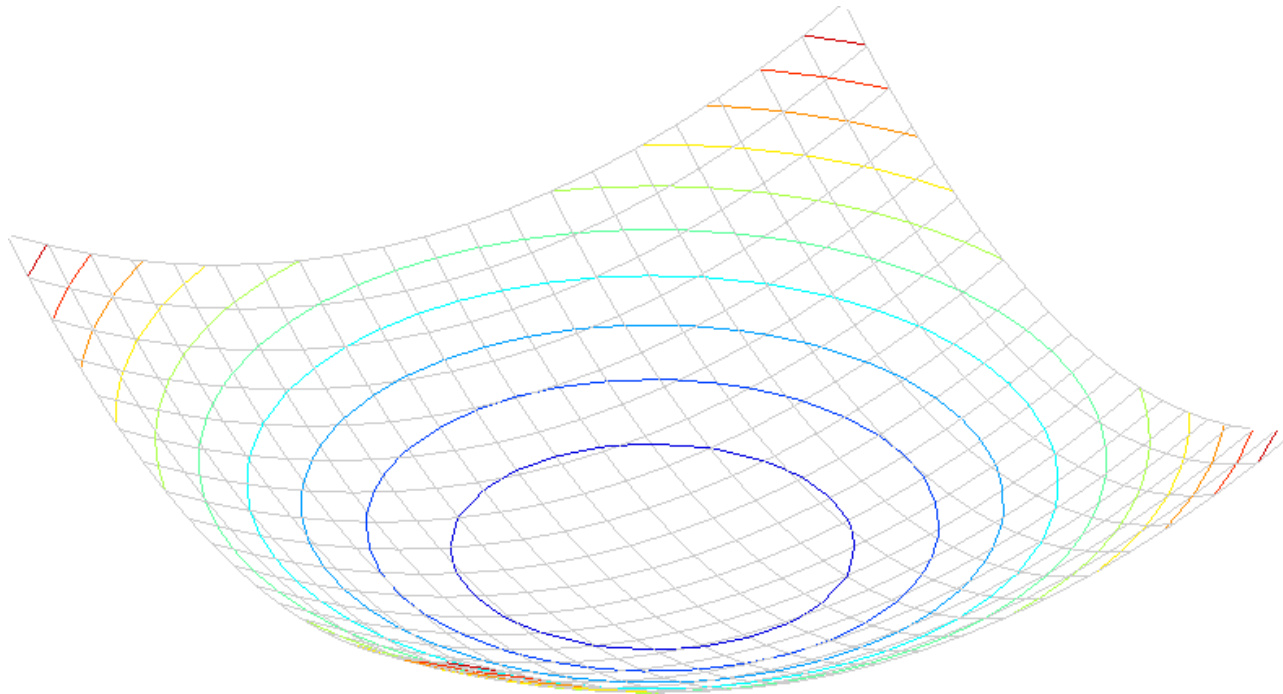


Interpreting the Second Moment Matrix

Let's take horizontal "slices" of our approximation of $E(u, v) = c$ (constant):

Each colored line in the diagram below is where $\begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix} = \text{constant}$

This is the equation of an ellipse.



$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

When you set $E(u, v) = c$ (a constant), those (u, v) points form a **level set** or "isocontour." For the **quadratic approximation**, each level set of

$$(u, v) \mapsto (u \ v) M \begin{pmatrix} u \\ v \end{pmatrix}$$

is a **2D ellipse** in the (u, v) plane.

- If M has two **small** eigenvalues, it means you can shift (u, v) quite a bit in any direction and E will not grow too quickly (a "flat" area in the image). The resulting ellipse is large.
- If M has **one large** and **one small** eigenvalue, you get a very **elongated ellipse** (characteristic of an edge: big change in one direction, little change in the perpendicular direction).
- If **both eigenvalues are large**, you cannot move much in **any** direction without making E big (this indicates a corner, so the ellipse is smaller and more "circular").

Linear Algebra: Recap

Because M is symmetric and positive semidefinite, it can be diagonalized. Let Q be the orthogonal matrix of eigenvectors and Λ the diagonal matrix of eigenvalues λ_1 and λ_2 . That is:

$$M = Q\Lambda Q^T, \quad \text{with} \quad \Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}.$$

If we change coordinates to (u', v') aligned with the eigenvectors (i.e., $\begin{pmatrix} u \\ v \end{pmatrix} = Q \begin{pmatrix} u' \\ v' \end{pmatrix}$), the quadratic form becomes:

$$E(u, v) = \lambda_1 u'^2 + \lambda_2 v'^2.$$

Setting $E(u, v) = c$ gives:

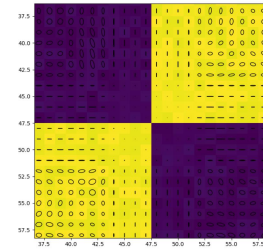
$$\lambda_1 u'^2 + \lambda_2 v'^2 = c.$$

Dividing through by c leads to:

$$\frac{u'^2}{c/\lambda_1} + \frac{v'^2}{c/\lambda_2} = 1,$$

which is the standard equation of an ellipse with semi-axes lengths $\sqrt{c/\lambda_1}$ and $\sqrt{c/\lambda_2}$.

Interpreting the Second Moment Matrix



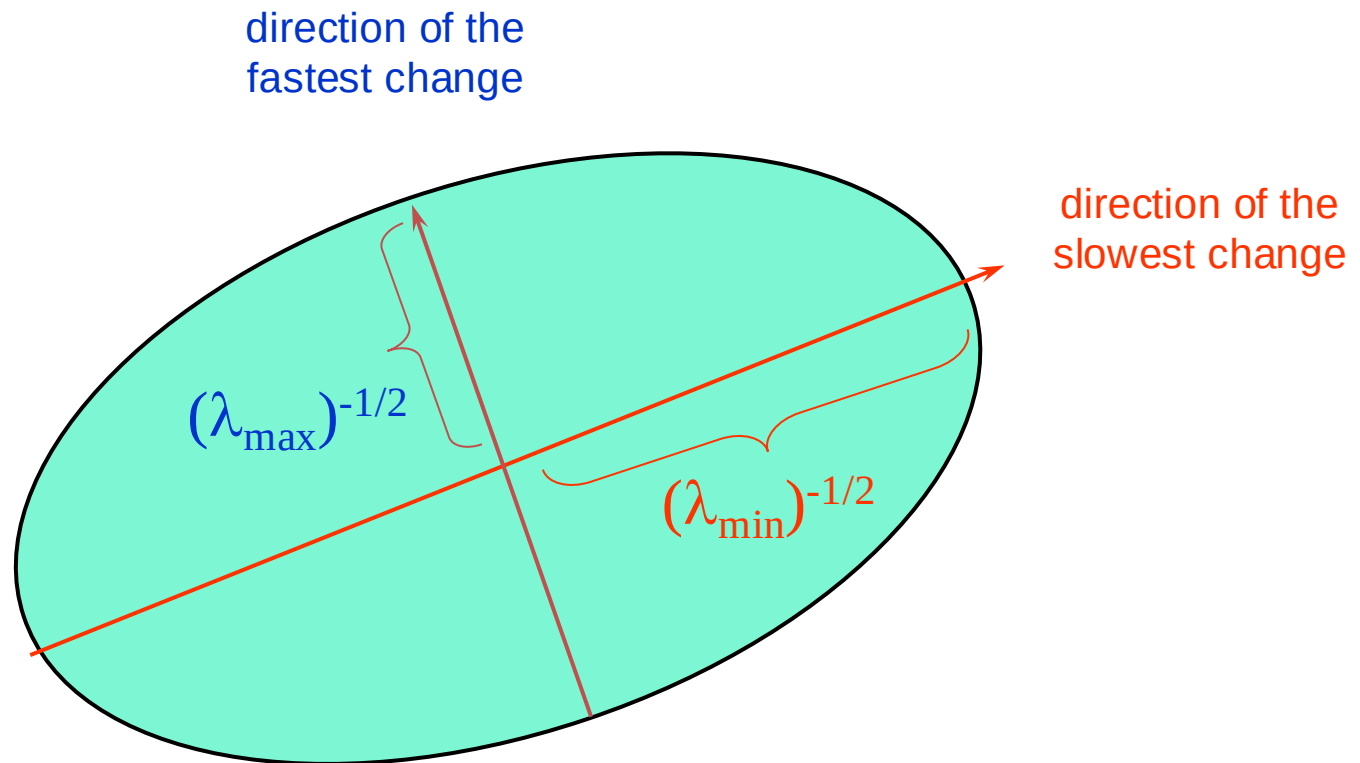
- **Axes of the ellipse = eigenvectors:** They indicate the directions in which $E(u, v)$ (or equivalently the local second-moment matrix) increases fastest or slowest.
- **Lengths of the axes = inversely related to the eigenvalues:** The eigenvalue tells you *how fast* E grows if you move in that eigenvector's direction. A large eigenvalue means a **steep** direction of change, leading to a *short* axis in the ellipse. A small eigenvalue means a **gentle** direction, so the ellipse is *long* along that axis.

So when you see one axis much larger than the other:

1. **Short axis $\Rightarrow$ large eigenvalue $\Rightarrow$ strong gradient** in that direction.
2. **Long axis $\Rightarrow$ small eigenvalue $\Rightarrow$ weak gradient** in that direction.

Interpreting the second moment matrix

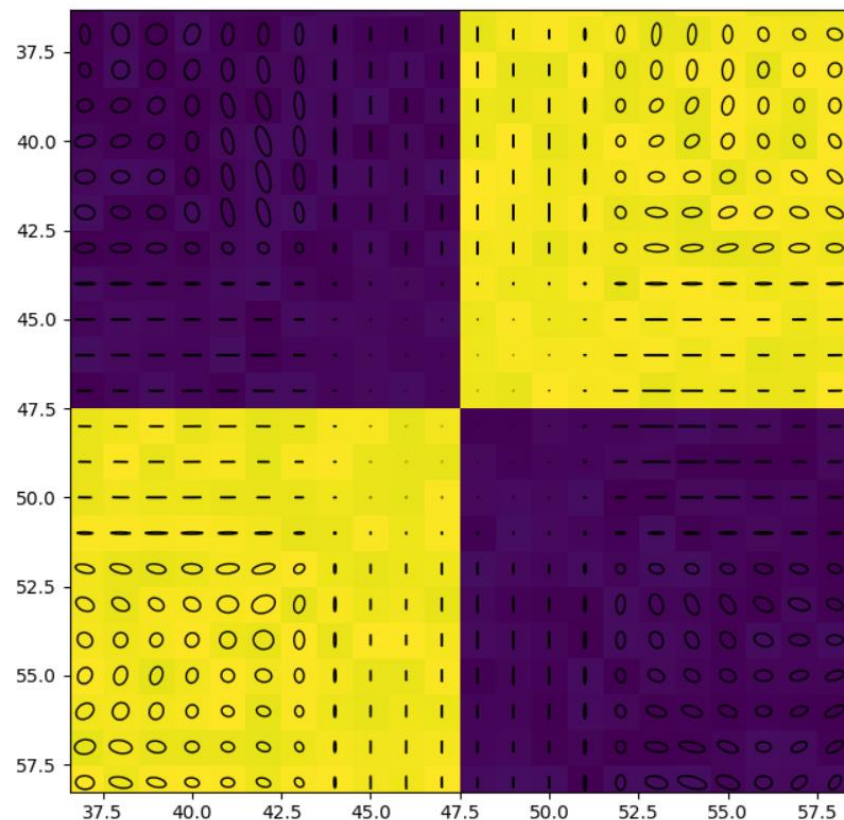
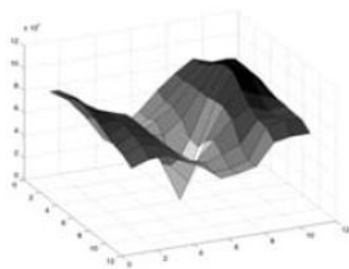
$$M = Q\Lambda Q^T, \quad \text{with} \quad \Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$$



Interpreting the Second Moment Matrix

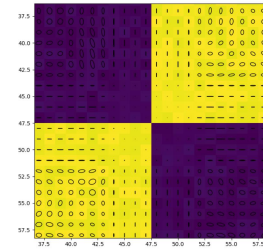
Simple checkerboard.

- 1 corner
- 4 edges
- 4 flat regions



Note: Edges show 'wide' response because image derivatives were blurred by Gaussian $\sigma = 1$ before visualizing.

Interpreting the Second Moment Matrix

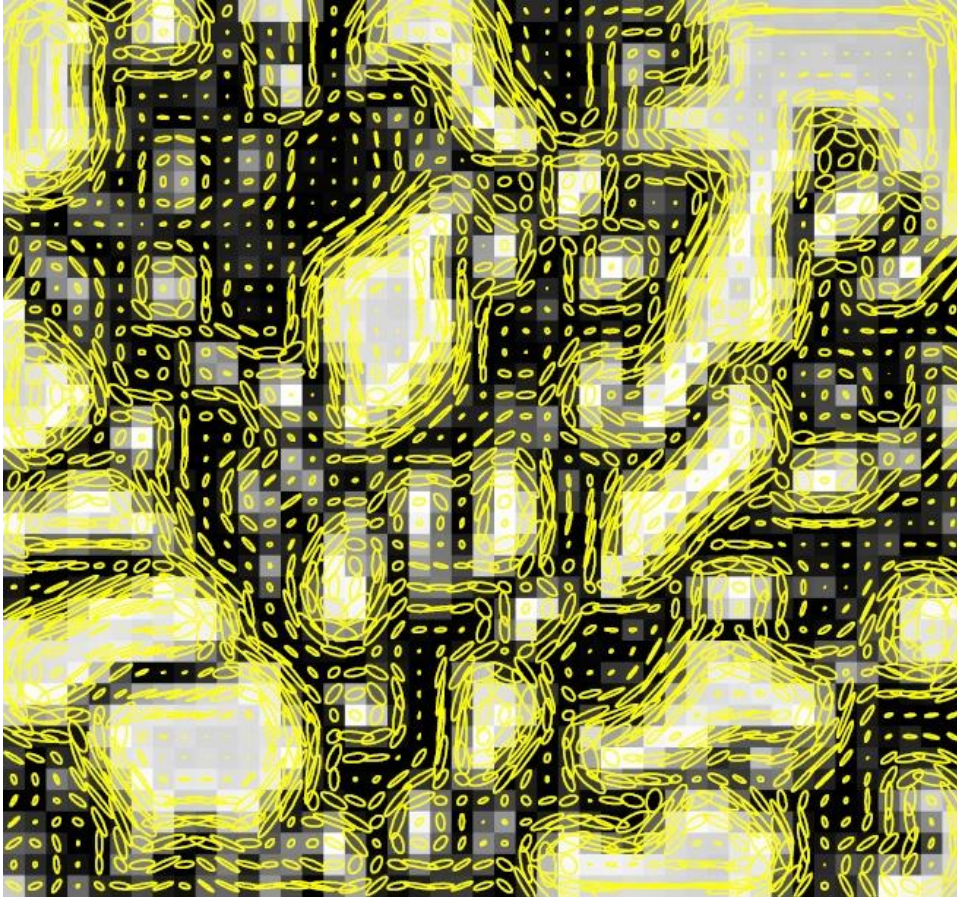


- **Axes of the ellipse = eigenvectors:** They indicate the directions in which $E(u, v)$ (or equivalently the local second-moment matrix) increases fastest or slowest.
- **Lengths of the axes = inversely related to the eigenvalues:** The eigenvalue tells you *how fast* E grows if you move in that eigenvector's direction. A large eigenvalue means a **steep** direction of change, leading to a *short* axis in the ellipse. A small eigenvalue means a **gentle** direction, so the ellipse is *long* along that axis.

So when you see one axis much larger than the other:

1. **Short axis $\Rightarrow$ large eigenvalue $\Rightarrow$ strong gradient** in that direction.
2. **Long axis $\Rightarrow$ small eigenvalue $\Rightarrow$ weak gradient** in that direction.

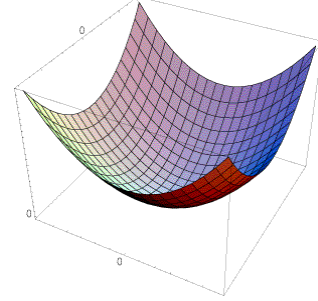
Interpreting the Second Moment Matrix



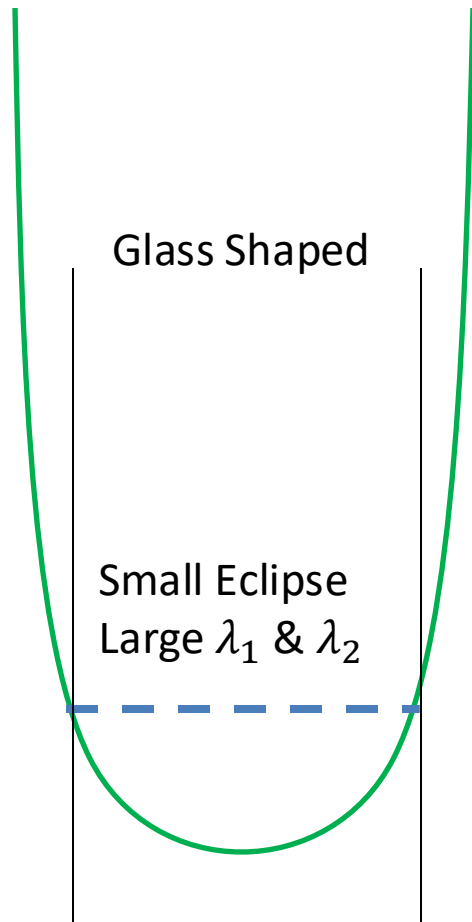
For corneriness,
we only care about
the 'steepness',
not the rotation.

So, we can ignore
the eigen vectors.

Illustration



$E(u,v)$ is locally approximated. The cross-section assumes the constant and same $E(u,v)$ for both the images



Glass Shaped

Small Eclipse
Large λ_1 & λ_2

Corner

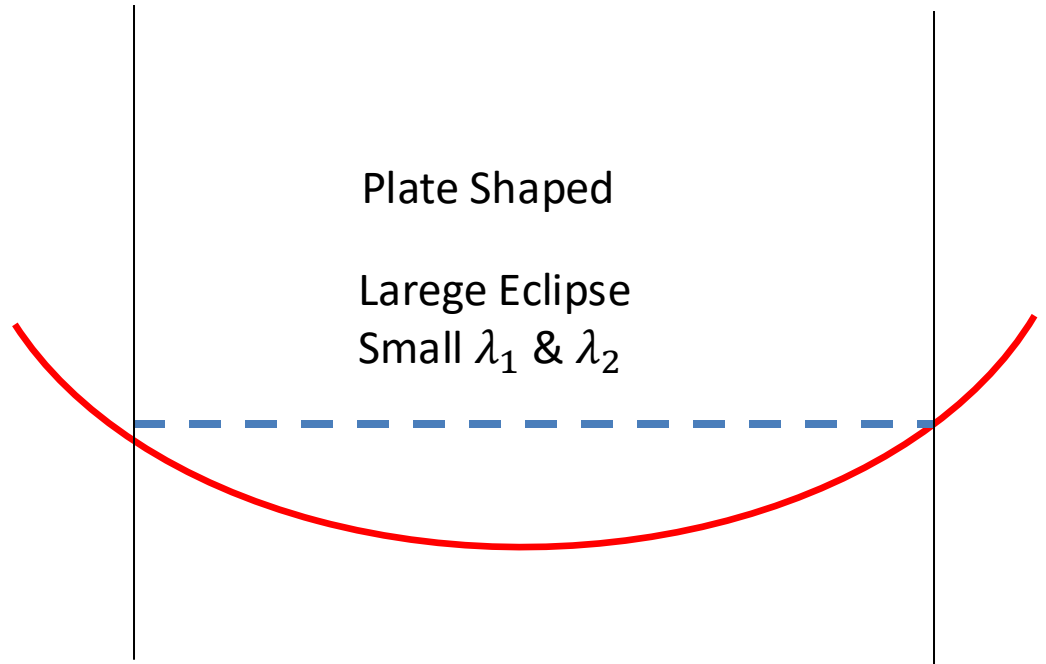
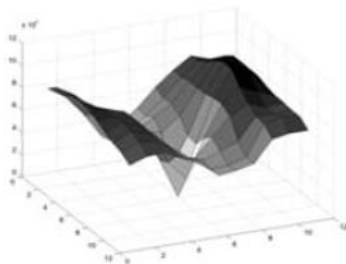
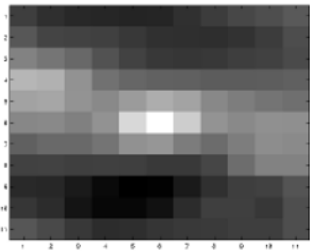


Plate Shaped

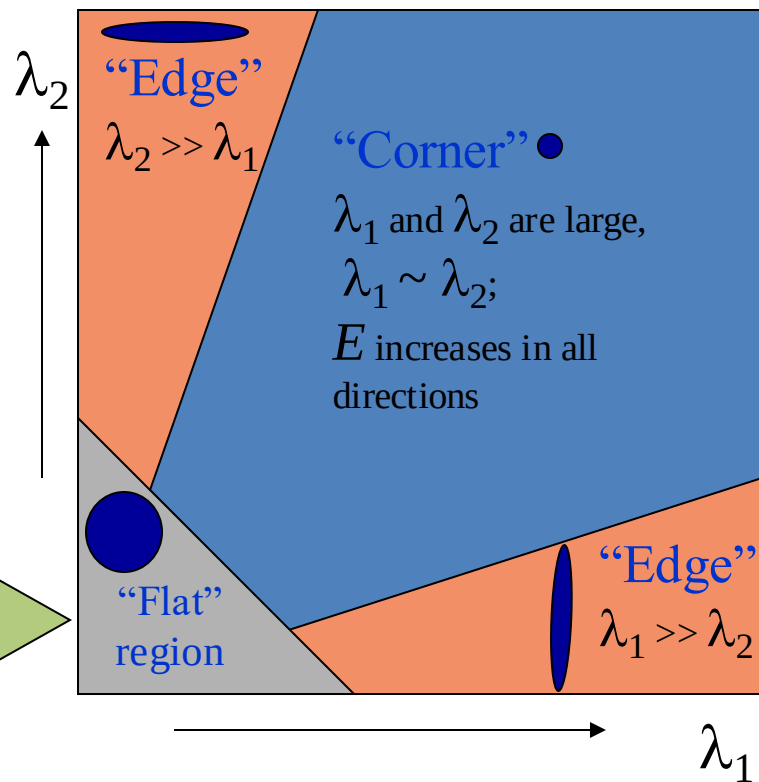
Large Eclipse
Small λ_1 & λ_2

Flat Region

Classification of image points using eigenvalues of M



λ_1 and λ_2 are small;
 E is almost constant in all
directions

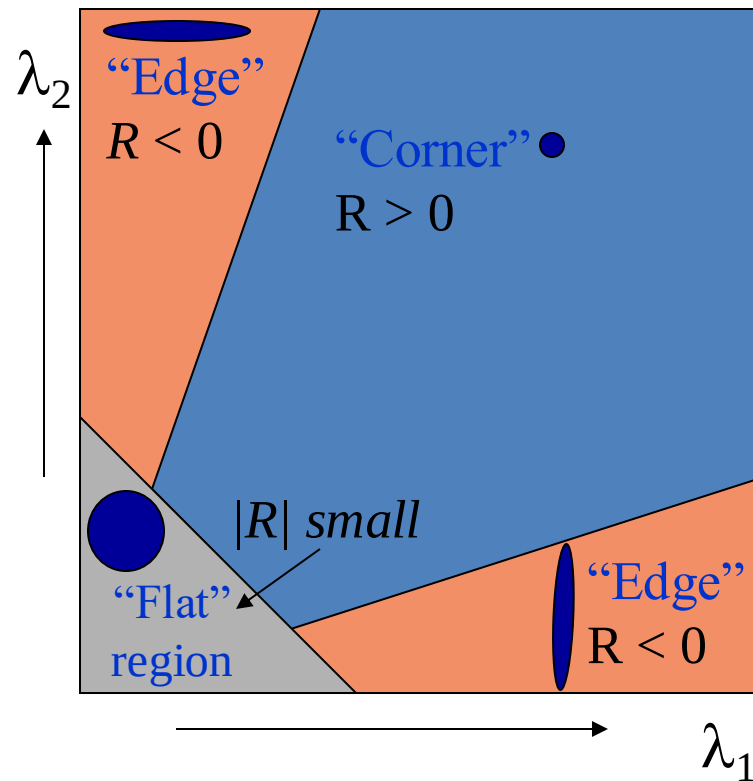


Classification of image points using eigenvalues of M

Cornerness score:

$$R = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

α : some constant (~ 0.04 to 0.06)



Linear Algebra: Recap

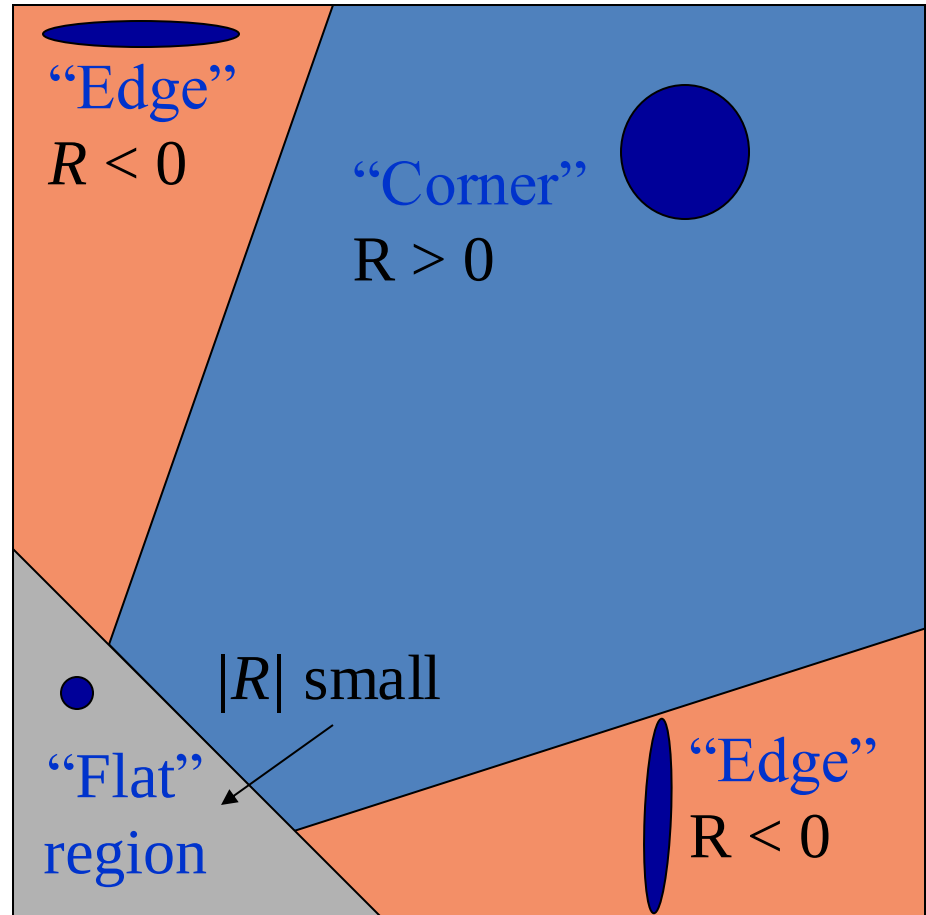
Determinant of a square matrix is the *product of all eigenvalues*.

Trace of a square matrix is the *sum of its eigenvalues*.

Corner response function

$$R = \det(M) - \alpha \operatorname{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

α : constant (0.04 to 0.06)



Classification of image points using eigenvalues of M

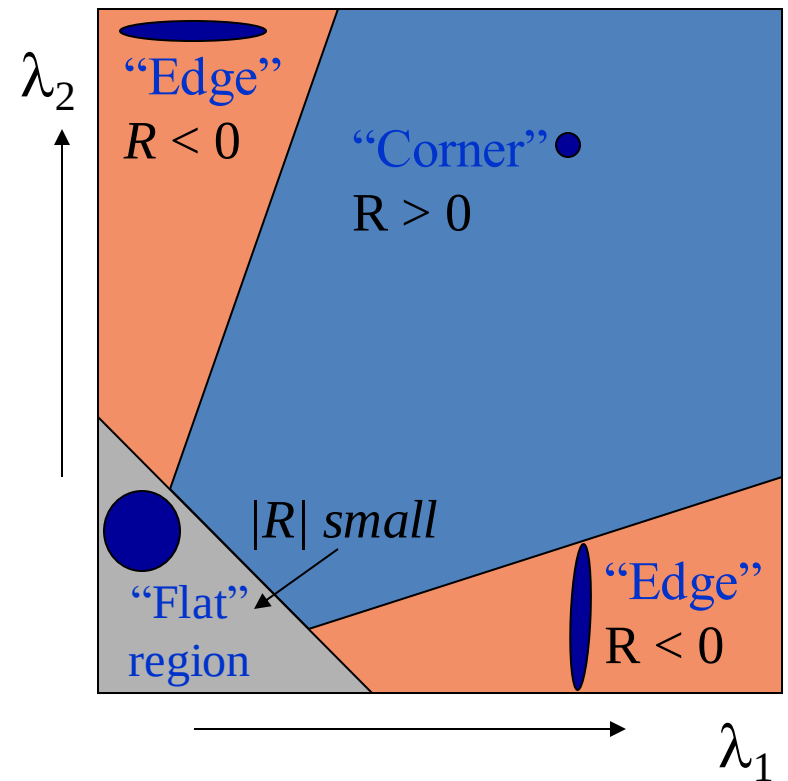
Cornerness score:

$$R = \lambda_1 \lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

α : some constant (~ 0.04 to 0.06)

$$R = \det(M) - \alpha \text{trace}(M)^2$$

Avoids explicit eigenvalue computation!



Harris Corner Detector

- 1) Compute M matrix for each pixel to recover *corneriness* score C .

Note: We can find M purely from the per-pixel image derivatives!

- 2) Threshold to find pixels which give large corner response
($R > \text{threshold}$).

- 3) Find the local maxima pixels, i.e., non-maxima suppression.

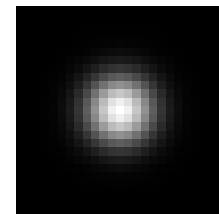
Weighting the derivatives

- In practice, using a simple window W doesn't work too well

$$M = \sum_{x,y} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Instead, we'll *weight* each derivative value based on its distance from the center pixel

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$



$w(x,y)$

Harris Corner Detector

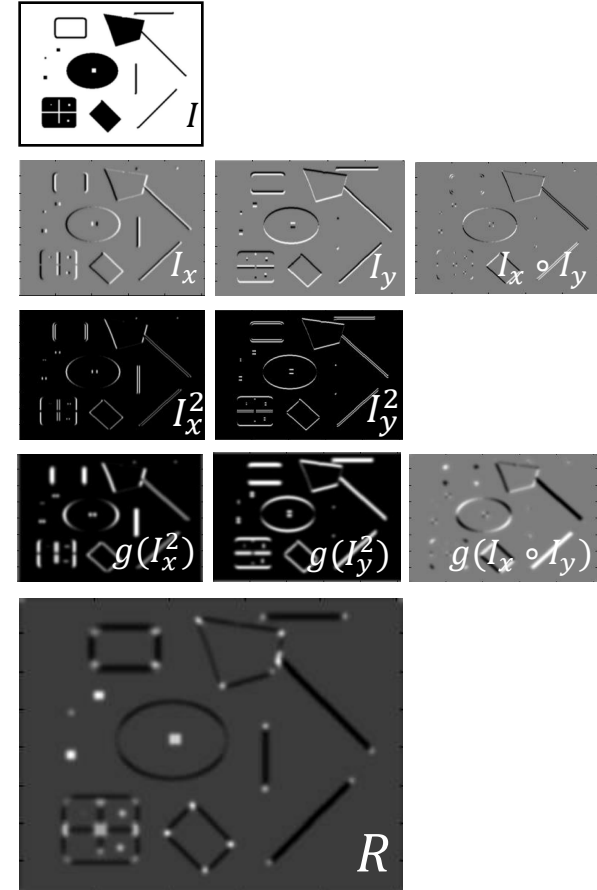
James Hays

- Compute image discrete first derivatives.
- Compute products of derivatives to use in M
- Create M via window function: Gaussian filter $g()$ with σ
 $g(I_x^2), g(I_y^2), g(I_x \circ I_y)$
- Compute cornerness

$$R = \det(M) - \alpha \text{trace}(M)^2 = g(I_x^2) \circ g(I_y^2) - g(I_x \circ I_y)^2$$

$$- \alpha [g(I_x^2) + g(I_y^2)]^2$$
- 5. Threshold on R to pick high cornerness
- 6. Non-maximal suppression to pick peaks.

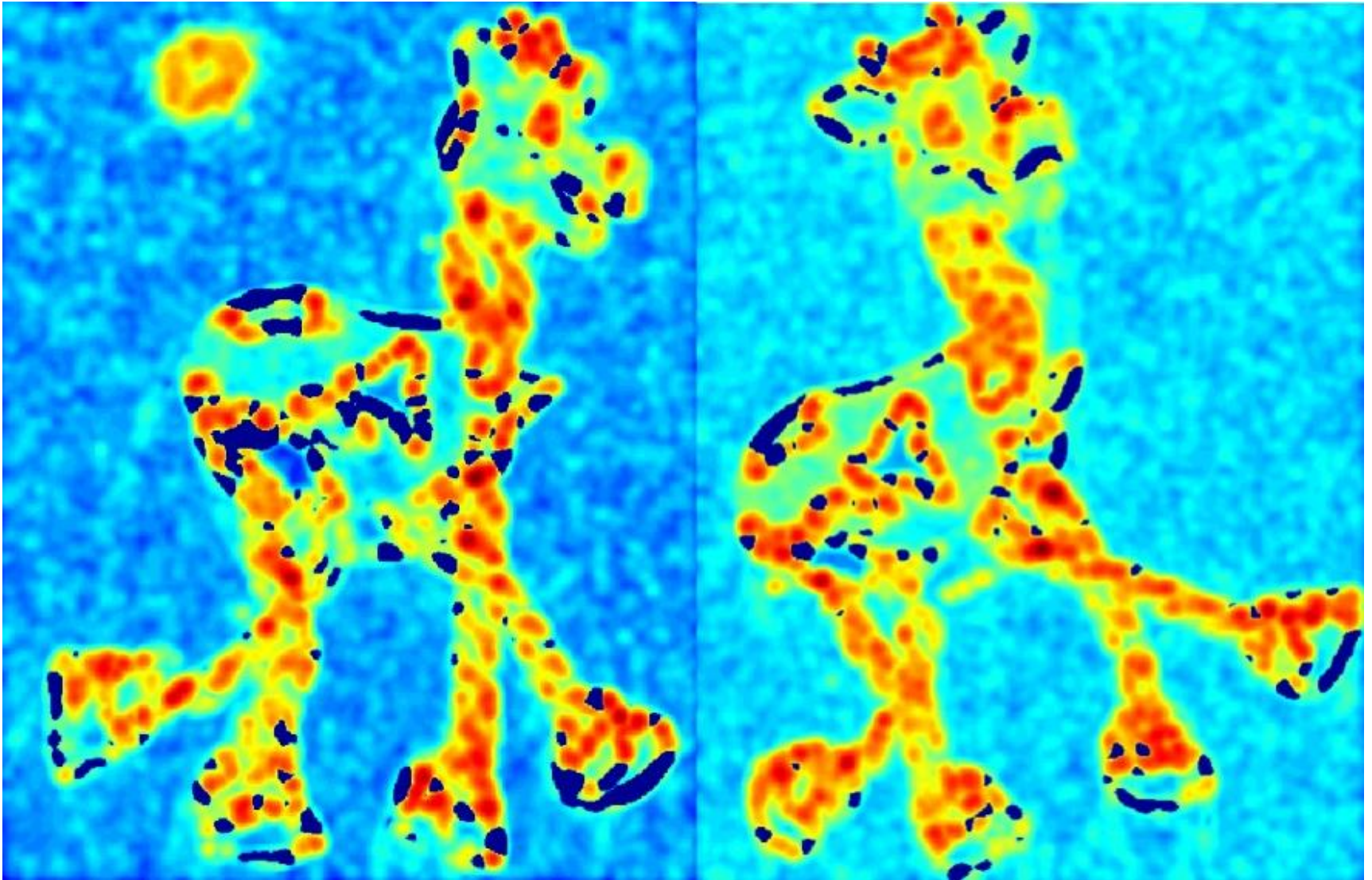
Reminder: $a \circ b$ is
Hadamard product
(element-wise
multiplication)



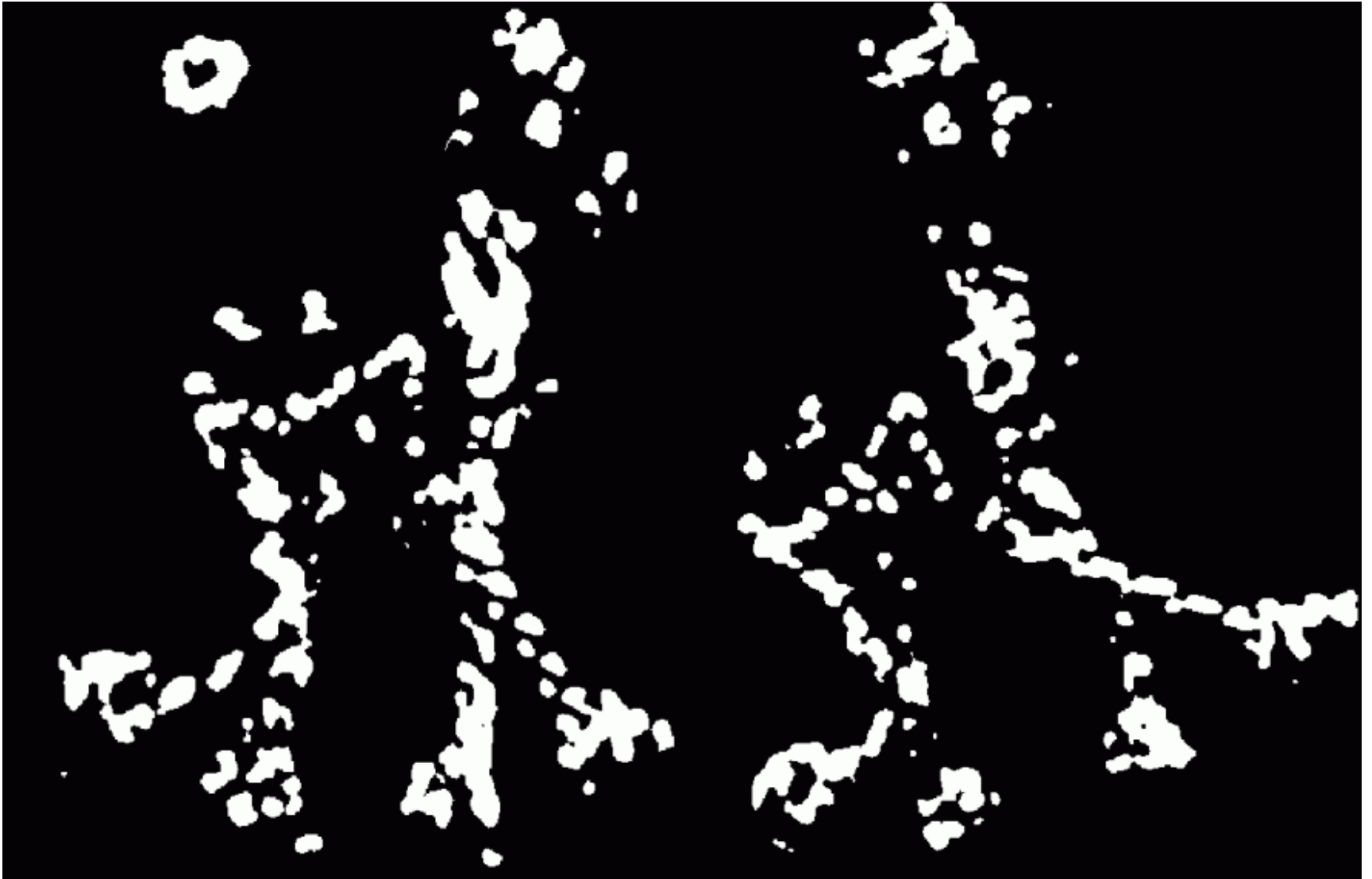
Harris detector example



Corner Response: R value
(red high, blue low)



Threshold ($R > \text{value}$)



Find local maxima of R (Non maxima Suppression)



Harris features (in red)



1. Harris & Stephens (1988)

$$R = \det(M) - \kappa (\text{trace}(M))^2$$

- Uses the **structure tensor** M to detect corners.
- κ (typically 0.04–0.06) controls sensitivity.

2. Shi & Tomasi (1994) / Kanade-Tomasi

$$R = \min(\lambda_1, \lambda_2)$$

- Selects features where the **smallest eigenvalue** is large.
- Ensures **good tracking** in optical flow.

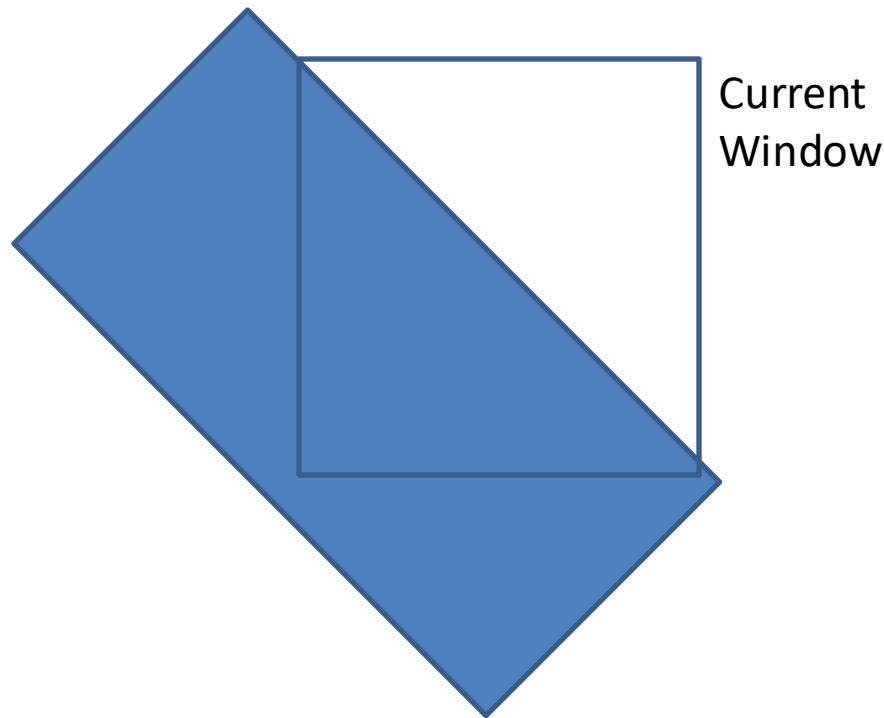
3. Noble (1989)

$$R = \frac{\det(M)}{\text{trace}(M) + \epsilon}$$

- Normalizes Harris response to avoid **parameter tuning**.
- ϵ prevents division by zero.

Harris Corners – Why so complicated?

- Can't we just check for regions with lots of gradients in the x and y directions?
 - No! A diagonal line would satisfy that criteria



OpenCV tutorial

- [Here](#)

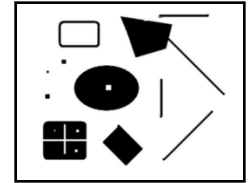
Harris corner detector

- 1) Compute M matrix for each image window to get their *cornerness* scores.
- 2) Find points whose surrounding window gave large corner response ($f > \text{threshold}$)
- 3) Take the points of local maxima, i.e., perform non-maximum suppression

C.Harris and M.Stephens. ["A Combined Corner and Edge Detector."](#)
Proceedings of the 4th Alvey Vision Conference: pages 147—151, 1988.

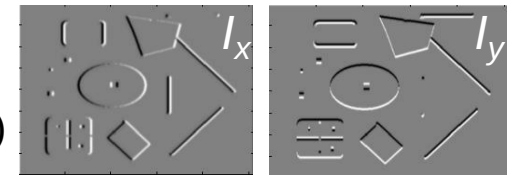
Harris Detector [Harris88]

- Second moment matrix

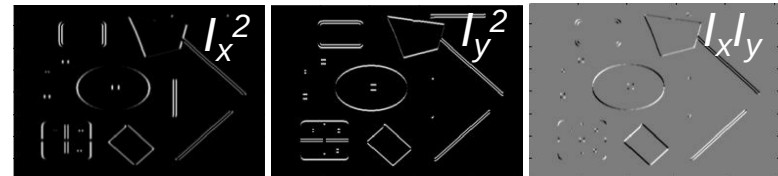


$$\mu(\sigma_I, \sigma_D) = g(\sigma_I) * \begin{bmatrix} I_x^2(\sigma_D) & I_x I_y(\sigma_D) \\ I_x I_y(\sigma_D) & I_y^2(\sigma_D) \end{bmatrix}$$

1. Image derivatives
(optionally, blur first)



2. Square of derivatives



3. Gaussian filter $g(\sigma_I)$



$$\det M = \lambda_1 \lambda_2$$

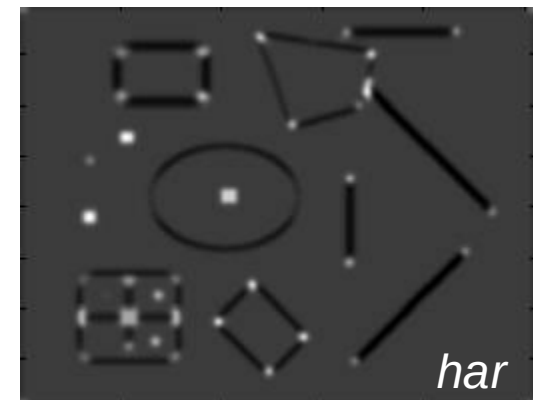
$$\text{trace } M = \lambda_1 + \lambda_2$$

4. Cornerness function – both eigenvalues are strong

$$har = \det[\mu(\sigma_I, \sigma_D)] - \alpha [\text{trace}(\mu(\sigma_I, \sigma_D))]^2 =$$

$$g(I_x^2)g(I_y^2) - [g(I_x I_y)]^2 - \alpha [g(I_x^2) + g(I_y^2)]^2$$

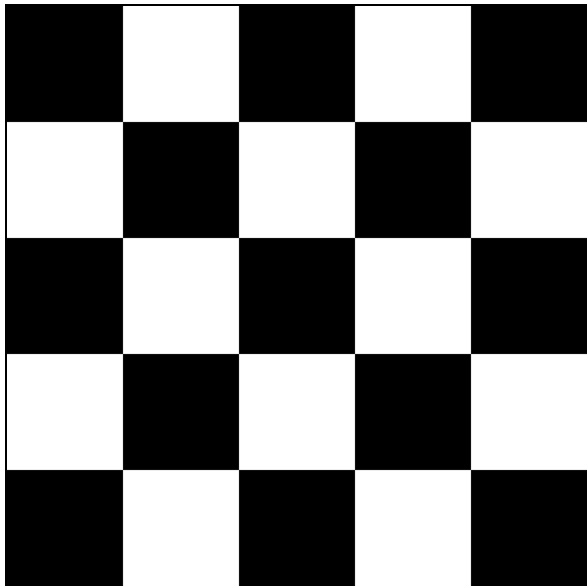
5. Non-maxima suppression



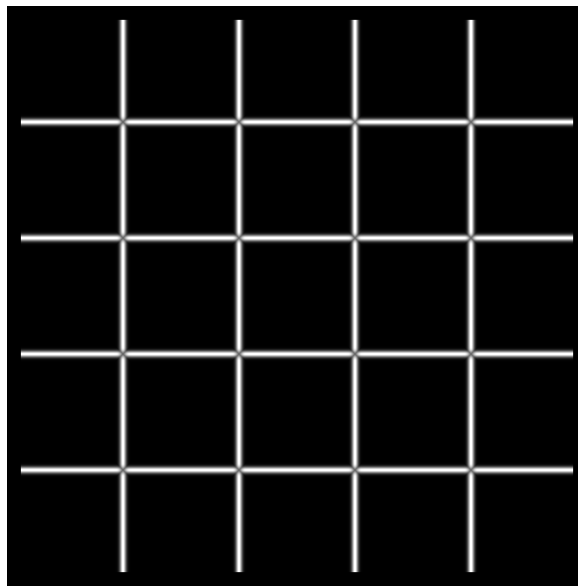
Corner detection summary + Last few steps

Here's what you do

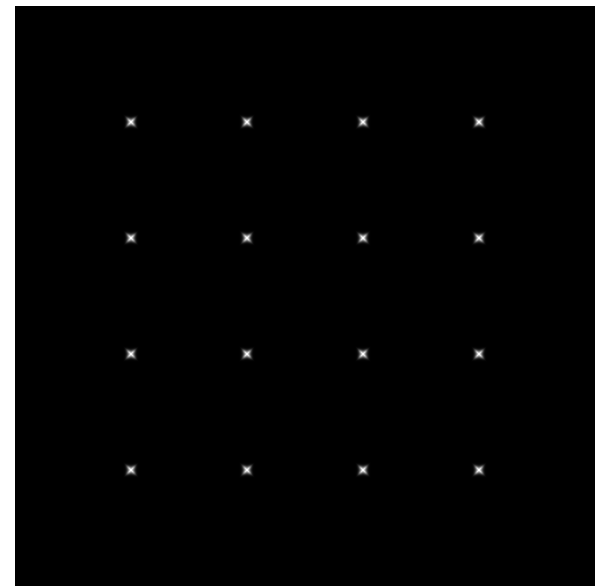
- Compute the gradient at each point in the image
- Create the M matrix from the entries in the gradient
- Compute the eigenvalues.
- Find points with large response ($\lambda_{\min} > \text{threshold}$)
- Choose those points where $\lambda_{\min}$ is a local maximum as features



I



$\lambda_{\max}$

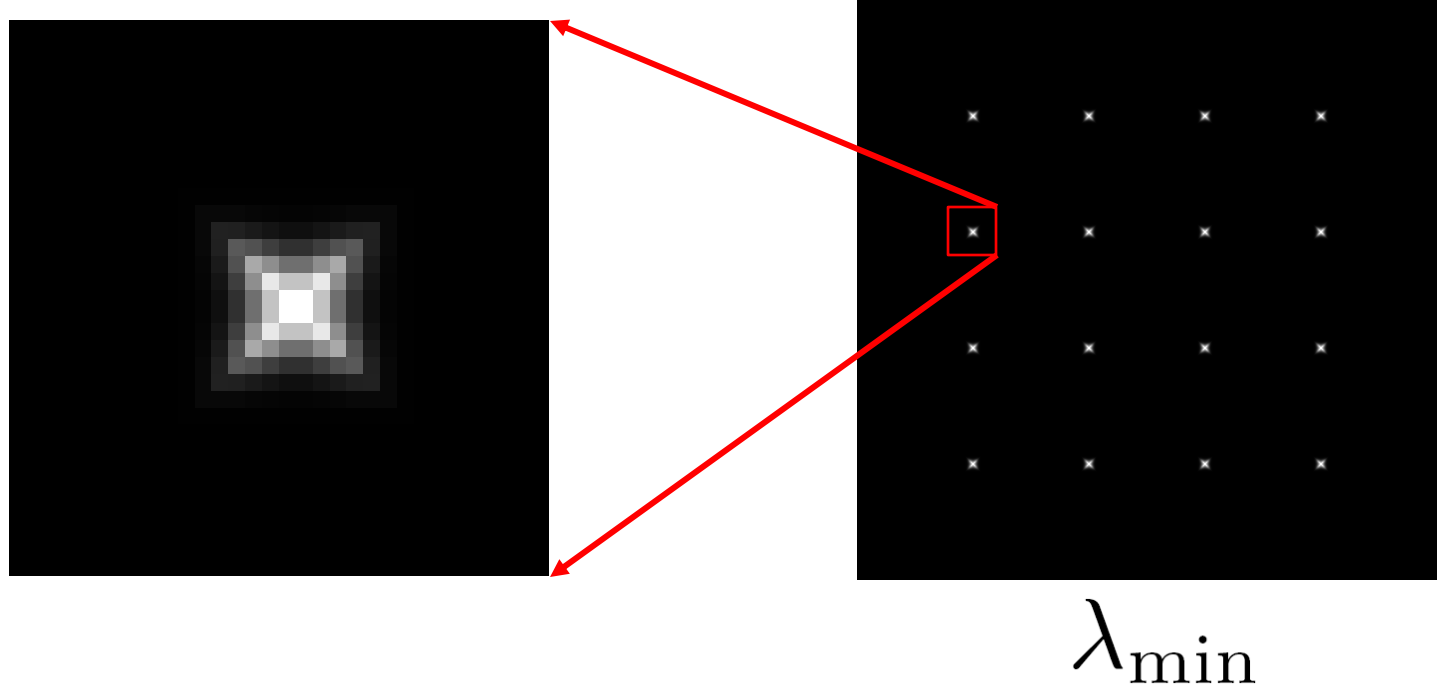


$\lambda_{\min}$

Corner detection summary

Here's what you do

- Compute the gradient at each point in the image
- Create the H matrix from the entries in the gradient
- Compute the eigenvalues.
- Find points with large response ($\lambda_{\min} > \text{threshold}$)
- Choose those points where $\lambda_{\min}$ is a local maximum as features



The Harris operator

